

PIXELCOINSLAUNCHER



HECHO POR: IKER MAGRO JUÁREZ

CURSO ACADÉMICO: 2024/2025

CONVOCATORIA: ORDINARIA

FECHA DE ENTREGA: 02/06/2025

TUTOR: JORGE LÓPEZ MELCHOR

MÓDULO PROFESIONAL: DESARROLLO DE APLICACIONES MULTIPLATAFORMA



RESUMEN

El presente proyecto, PixelCoinsLauncher, consiste en el desarrollo de una aplicación de escritorio en JavaFX cuyo propósito principal es incentivar a los usuarios a jugar videojuegos ofreciéndoles recompensas virtuales en función del tiempo jugado. La aplicación permite gestionar una biblioteca de juegos, ejecutar los títulos registrados y monitorizar cada sesión de juego para otorgar monedas virtuales. Estas monedas pueden canjearse por productos en una tienda interna, ofreciendo así una alternativa económica al modelo tradicional de compra de videojuegos.

Este proyecto surge como respuesta a una problemática común: los altos precios de los videojuegos en tiendas físicas y digitales. La solución propuesta combina gamificación y utilidad, creando un entorno donde el tiempo invertido jugando se traduce en recompensas. El público objetivo son jugadores que desean registrar su tiempo de juego y obtener beneficios sin incurrir en gastos adicionales.

Para el desarrollo se ha seguido una metodología basada en el ciclo de vida del software, incluyendo análisis de requisitos, diseño, codificación, pruebas y verificación. Se han utilizado tecnologías como Java, JavaFX, CSS, MySQL, y diversas librerías como JNativeHook y JNA para implementar funcionalidades como el sistema anti-AFK, que penaliza la inactividad del usuario durante las sesiones de juego.

La elaboración del proyecto ha requerido aproximadamente tres meses. Se ha evaluado mediante pruebas unitarias y de interfaz gráfica con JUnit y TestFX, validando tanto funcionalidades como experiencia de usuario. Como conclusión, se destaca la utilidad del sistema para el usuario final y el potencial de ampliación a plataformas móviles o web.

Palabras clave: JavaFX, videojuegos, sistema de recompensas, launcher, gamificación.



ABSTRACT

The present project, PixelCoinsLauncher, is a desktop application developed in JavaFX with the primary objective of encouraging users to play video games by offering virtual rewards based on their playtime. The application allows users to manage a personal game library, execute registered games, and track each gaming session to assign virtual coins. These coins can later be exchanged for products in an internal store, offering a cost-effective alternative to conventional video game purchasing models.

This project addresses a current issue: the high prices of video games in both physical and digital stores. The proposed solution merges gamification and utility, creating an environment where time spent playing translates into benefits. The target users are players who wish to record their gaming sessions and gain rewards without additional expenses.

The development followed a methodology based on the software development life cycle, including requirements analysis, design, coding, testing, and verification. Technologies used include Java, JavaFX, CSS, MySQL, and libraries such as JNativeHook and JNA to enable features like the anti-AFK system, which penalizes inactivity during gaming sessions.

The project development took approximately three months. It was evaluated through unit and UI testing using JUnit and TestFX, ensuring both feature correctness and user experience. In conclusion, the system proves useful for end-users and shows strong potential for future expansion into mobile or web platforms.

Keywords: JavaFX, video games, reward system, launcher, gamification.



ÍNDICE

1. JUSTIFICACIÓN.....	5
2. OBJETIVOS.....	5
3. METODOLOGÍAS DE DESARROLLO.....	6
3.1 ANÁLISIS DE REQUISITOS.....	6
3.2 DISEÑO DEL SISTEMA.....	6
3.3 DISEÑO DEL PROGRAMA.....	13
3.4 CODIFICACIÓN.....	34
3.5 EJECUCIÓN DE PRUEBAS.....	35
3.6 VERIFICACIÓN.....	42
3.7 MANTENIMIENTO.....	43
4. TECNOLOGÍAS.....	44
5. PLANIFICACIÓN DEL PROYECTO.....	45
6. DESARROLLO Y EJECUCIÓN DEL PROYECTO.....	46
7. CONCLUSIONES.....	47
8. LÍNEAS DE INVESTIGACIÓN Y DESARROLLO FUTURAS.....	48
9. BIBLIOGRAFÍA.....	48
10. EXTRAS.....	49
10.1 ANÁLISIS DE MERCADO.....	49
10.2 SEGURIDAD Y PROTECCIÓN DE DATOS.....	49
10.3 DESPLIEGUE.....	50
11. ANEXO - MANUAL DE USUARIO.....	53



1. JUSTIFICACIÓN

El proyecto surge como respuesta al alto coste de los videojuegos en el mercado actual. La mayoría de los jugadores no dispone de recursos económicos suficientes para acceder a títulos de forma habitual, lo cual representa una barrera. PixelCoinsLauncher propone una solución innovadora: recompensar el tiempo jugado con monedas virtuales que pueden usarse para adquirir productos dentro de una tienda interna.

Esto permite democratizar el acceso a contenidos, a la vez que incentiva el uso activo de videojuegos. Su aplicabilidad práctica es clara, y su desarrollo ha potenciado competencias clave como diseño de interfaces, gestión de sesiones, almacenamiento en base de datos y control de versiones.

2. OBJETIVOS

Objetivo principal: desarrollar una aplicación de escritorio de JavaFX que pueda permitir a los usuarios gestionar una biblioteca de juegos, registrar sesiones de juego, ganar monedas virtuales en función del tiempo jugado y se acceda a una tienda interna.

Objetivos específicos:

1. Implementar un sistema de registro y autenticación de usuarios con validaciones y persistencia en base de datos.
2. Permitir a cada usuario agregar, listar y eliminar juegos de su biblioteca personal.
3. Registrar y almacenar sesiones de juego, incluyendo tiempo jugado y sistema anti-AFK para evitar la acumulación injusta de monedas.
4. Calcular y asignar monedas virtuales a los usuarios según el tiempo efectivo jugado.
5. Desarrollar una tienda interna donde los usuarios puedan visualizar y, en el futuro, canjear productos usando sus monedas.
6. Desarrollar una interfaz gráfica moderna, intuitiva y animada utilizando JavaFX.
7. Garantizar la persistencia de toda la información relevante (usuarios, juegos, sesiones, monedas) en una base de datos.



3. METODOLOGÍAS DE DESARROLLO

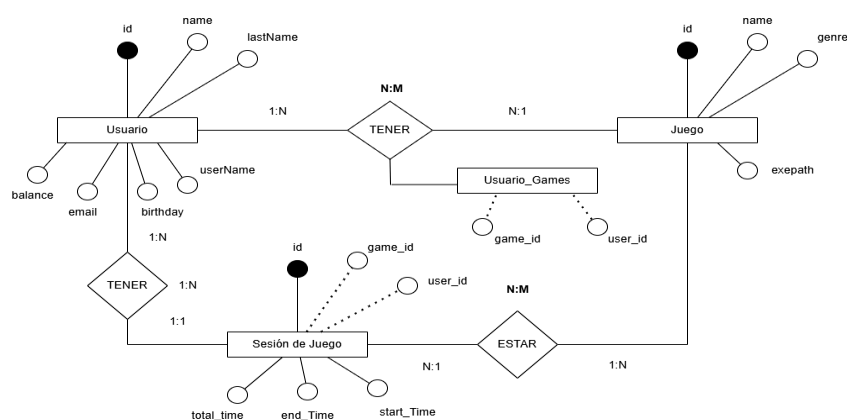
3.1 ANÁLISIS DE REQUISITOS

- Requisitos funcionales:
 - Registro y autenticación de usuarios.
 - Gestión de biblioteca de juegos por usuario.
 - Ejecución de juegos y registro de sesiones.
 - Sistema de monedas virtuales basado en el tiempo jugado.
 - Tienda interna para gastar monedas.
 - Sistema anti-AFK para evitar la acumulación de monedas sin actividad.
 - Interfaz gráfica moderna y animada con JavaFX.
 - Persistencia de datos en base de datos.
- Requisitos no funcionales:
 - Interfaz intuitiva.
 - Persistencia de datos segura.

3.2 DISEÑO DEL SISTEMA

Base de datos

DIAGRAMA DEL MODELO ENTIDAD-RELACIÓN



Para poder empezar a explicar el modelo entidad-relación necesitamos saber y conocer las distintas entidades:



- Usuario: aquella entidad que va representar los datos personales del registro e inicio de sesión en la base de datos. Esta entidad tiene los siguientes registros:
 - id: identificador único del usuario (clave primaria).
 - name: nombre del usuario.
 - lastName: apellidos del usuario.
 - userName: nombre de usuario.
 - email: correo electrónico del usuario.
 - birthday: día de nacimiento del usuario.
 - balance: monedas que tiene el usuario.
- Juego: aquella entidad que va representar los datos de la biblioteca de juegos donde se va almacenar todos los juegos que el usuario añada, elimine y que ejecute. Esta entidad tiene los siguientes registros:
 - id: identificador único del juego (clave primaria).
 - name: nombre del juego.
 - genre: género del juego.
 - exepath: ruta del archivo .exe del juego.
- Sesión de juego: aquella entidad que va representar los datos de la monitorización del tiempo jugado de una sesión de juego de un juego que ha jugado el usuario. Esta entidad tiene los siguientes registros:
 - id: identificador único de la sesión de juego (clave primaria).
 - start_time: fecha y hora sobre el comienzo de la sesión de juego.
 - end_time: fecha y hora sobre la finalización de la sesión de juego.
 - total_time: tiempo jugado total representado en segundos.
 - user_id: identificador del usuario (clave foránea que apunta a entidad Usuario).
 - game_id: identificador del juego (clave foránea que apunta a entidad Juego).

Ya comprendido las entidades, se necesita explicar por qué se dan las siguientes relaciones:

- TENER (Usuario-Juego): esta relación representa que un usuario puede tener varios juegos y un juego puede estar asociado a varios usuarios. Por lo tanto, esta relación entre estas dos entidades se consideran una relación de muchos a muchos (N:M). Esto quiere decir, que necesita que exista una tabla intermedia, ya que este tipo de relaciones no se puede modelar directamente, debido a que en una base de datos relacional como MySQL no permite relaciones N:M. Entonces, se crearía la tabla "Usuario_Games" cogiendo los identificadores del

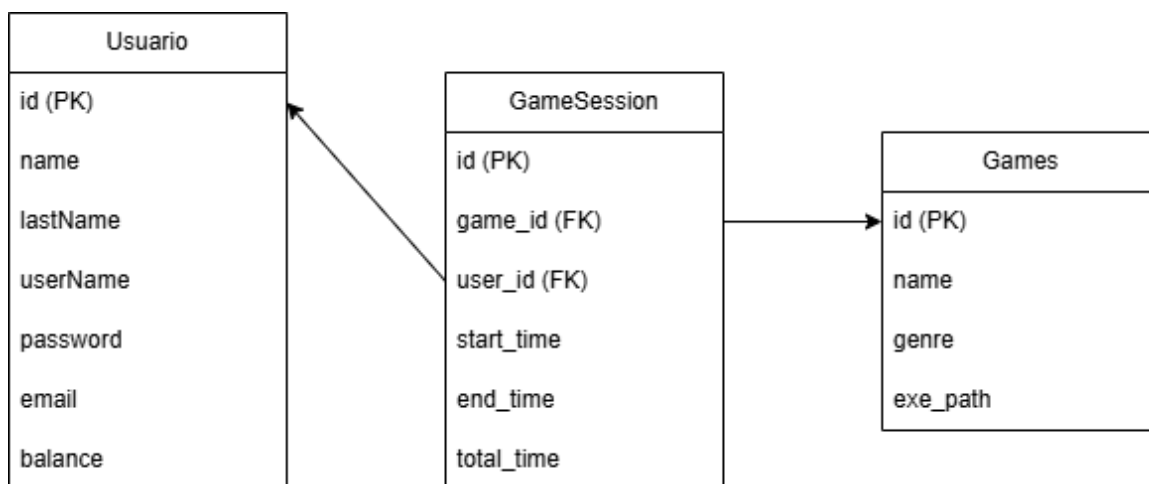


- usuario (user_id) y juego (game_id). A parte de que necesita eliminarse y actualizarse en cascada, ya que servirá para eliminar o actualizar aquellos datos que estén asociados a las tablas.
- TENER (Usuario-Sesión de juego): esta relación representa que un usuario puede tener varias sesiones, pero cada sesión sólo pertenece a un único usuario. Por lo tanto, se considera una relación de uno a muchos (1:N).
 - ESTAR (Sesión de juego-Juego): esta relación representa que una sesión de juego está asociada a un único juego, mientras que un juego puede tener muchas sesiones de juego. Por lo tanto, se considera una relación de muchos a muchos (N:M).

Explicado las entidades, podemos decir que este sistema se gestiona a través de que los usuarios pueden poseer múltiples juegos, en la cual, cada vez que juegan, se va registrar una sesión que va indicar cuándo comenzó, cuándo terminó y cuánto duró en la base de datos. En cada sesión va estar ligada a un único usuario y a un único juego.

Las claves primarias representan las bolas negras, mientras que las claves foráneas representan las líneas discontinuas.

DIAGRAMA DEL MODELO RELACIONAL



Tras realizar todos los diagramas relacionados con la base de datos, hay que crear la base de datos. Para ello, se debe usar un servicio web para almacenar de manera gratuita la base de datos en la nube, Railway. Es un servicio web que proporciona y gestiona tu servicio web actuando como host del servidor de tu servicio o aplicación. En mi caso, fue útil para guardar mi base de datos MySQL en la nube. Esto me proporcionó herramientas para tener la conexión con mi base de datos mediante un host desde



DBeaver, por lo que podía manipular, crear o eliminar tablas o registros en la base de datos.

General Driver properties SSH SSL + Network configurations...

Server

Connect by: ☒ Host ☐ URL

URL: jdbc:mysql://nozomi.proxy.rlwy.net:50148/railway

Server Host: nozomi.proxy.rlwy.net Port: 50148

Database: railway

Authentication (Database Native)

Nombre de usuario: root

Contraseña: ☒ Save password

Imagen representativa de la configuración de la conexión de la base de datos con Railway

REPRESENTACIÓN DE LA BASE DE DATOS EN DBEAVER

```
● CREATE TABLE Usuario (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(30) NOT NULL,
    lastName VARCHAR(30) NOT NULL,
    userName VARCHAR(30) NOT NULL,
    password VARCHAR(30) NOT NULL,
    email VARCHAR(30) NOT NULL,
    birthday DATE NOT NULL,
    balance INT DEFAULT 0
);

● CREATE TABLE games(
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    genre VARCHAR(100) NOT NULL,
    exePath VARCHAR(100) NOT NULL
);

● CREATE TABLE GameSession(
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    game_id BIGINT NOT NULL,
    user_id BIGINT NOT NULL,
    start_time DATETIME NOT NULL,
    end_time DATETIME NOT NULL,
    total_time INT NOT NULL,
    FOREIGN KEY (game_id) REFERENCES games(id),
    FOREIGN KEY (user_id) REFERENCES Usuario(id)
);

● CREATE TABLE Usuario_Games (
    user_id BIGINT NOT NULL,
    game_id BIGINT NOT NULL,
    PRIMARY KEY (user_id, game_id),
    FOREIGN KEY (user_id) REFERENCES Usuario(id) ON DELETE CASCADE,
    FOREIGN KEY (game_id) REFERENCES games(id) ON DELETE CASCADE
);
```



REPRESENTACIÓN DE LA BASE DE DATOS EN RAILWAY

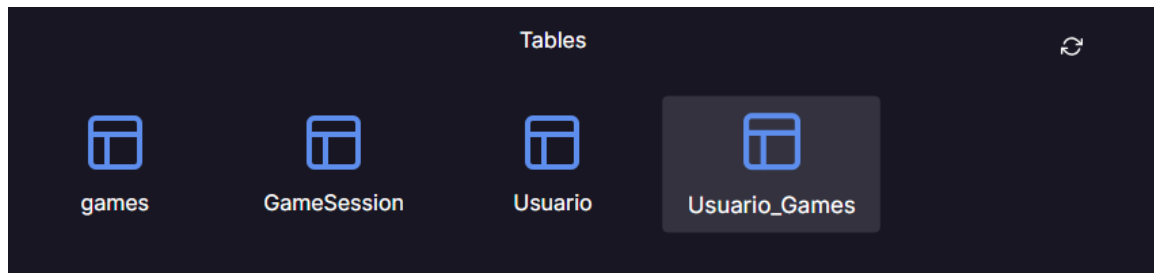


Imagen representativa de las tablas creadas en DBeaver representadas gracias a la conexión de la base de datos en Railway.

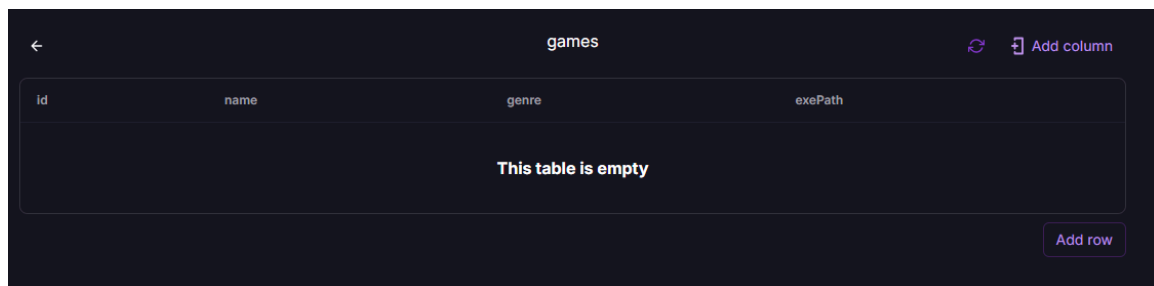


Imagen representativa de la tabla “games” con sus correspondientes atributos.

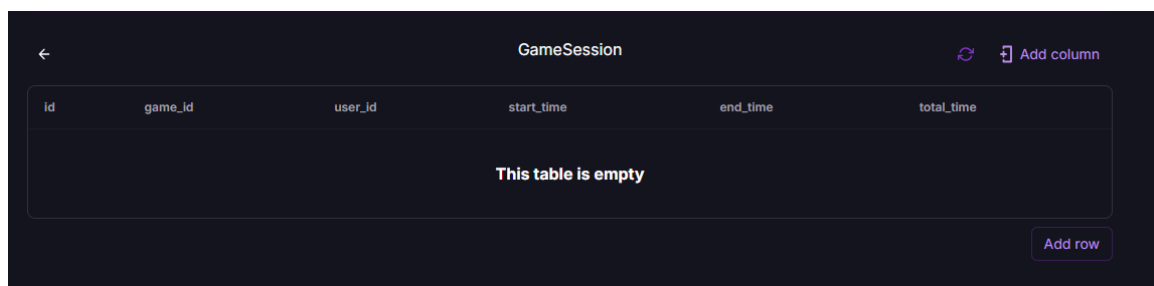


Imagen representativa de la tabla “GameSession” con sus correspondientes atributos..

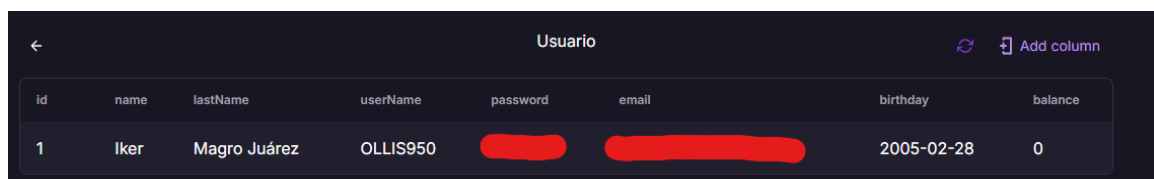


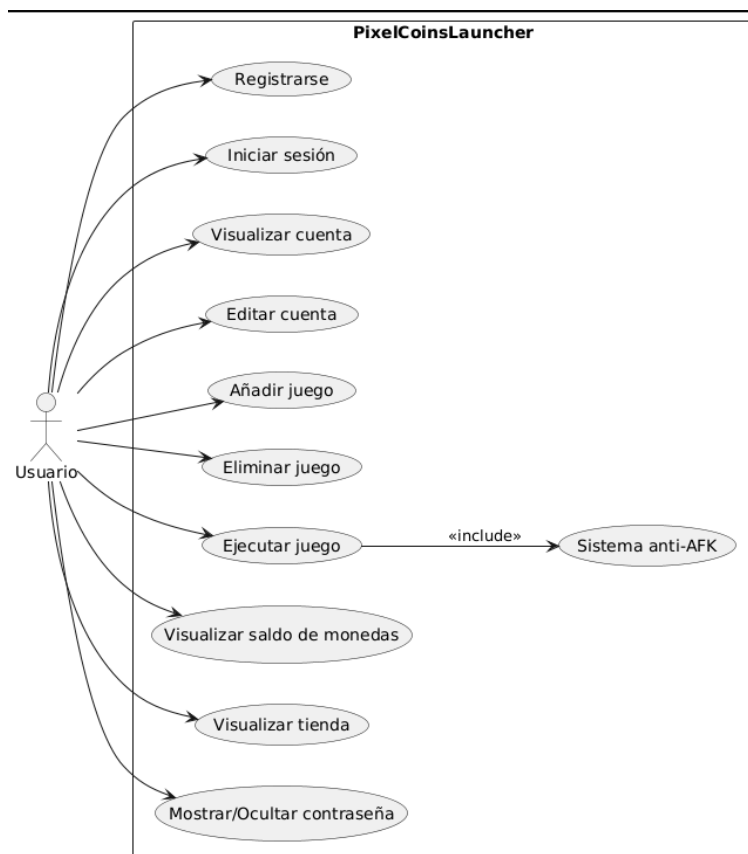
Imagen representativa de la tabla “Usuario” con sus correspondientes atributos.



Usuario_Games	
user_id	game_id
This table is empty	

Imagen representativa de la tabla intermedia “Usuario_Games” con sus correspondientes atributos.

Casos de uso





1. Registrarse

- Permite que un usuario cree una cuenta en el sistema.
- Acción inicial para acceder a otras funciones.

2. Iniciar sesión

- El usuario ya registrado puede ingresar con sus credenciales.
- Posiblemente conecta con la base de datos de usuarios.

3. Visualizar cuenta

- El usuario puede ver información de su cuenta, como nombre, correo, etc.

4. Editar cuenta

- Permite modificar datos personales como email, nombre o contraseña.

5. Añadir juego

- El usuario puede agregar un nuevo juego a su biblioteca dentro del launcher.

6. Eliminar juego

- Permite quitar un juego previamente añadido.

7. Ejecutar juego

- Opción para iniciar un juego desde el launcher.
- Este caso de uso incluye la función "Sistema anti-AFK". Este sistema se activa cuando el usuario hace clic en "Ejecutar juego".

8. Visualizar saldo de monedas

- Muestra el total de monedas virtuales que tiene el usuario en la plataforma.

9. Visualizar tienda

- Permite navegar por la tienda para comprar elementos con monedas virtuales.



10. Mostrar/Ocultar contraseña

- Funcionalidad útil en formularios de login o registro.
- Mejora la experiencia y seguridad al introducir la contraseña.

3.3 DISEÑO DEL PROGRAMA

La arquitectura del software sigue la arquitectura del Modelo-Vista-Controlador (MVC), es decir, un sistema que separa la lógica del negocio para poder conseguir una arquitectura modular y tener mejor ordenado el proyecto. Esta arquitectura se compone de la siguiente forma:

1. Interfaz gráfica (Vista):

La interfaz gráfica es aquello que el usuario va visualizar del software mediante componentes como botones, desplegables, campos de texto, etc. En este caso, la interfaz gráfica se desarrolla mediante Java gracias a la librería JavaFX. Por lo tanto, este proyecto se va componer de varias ventanas visuales en el que el usuario va interactuar:

- **com.proyect.MainWindow:** esta clase será la principal, donde se gestionan las ventanas de registro e inicio de sesión. Estas ventanas administran la autenticación y el acceso a la aplicación principal.



Imagen representativa de la ventana principal.



La correspondiente imagen muestra el menú principal de la aplicación, donde se puede acceder al registro de una cuenta o iniciar sesión si tenemos una cuenta registrada en la base de datos.

The image shows a window titled "FretComLauncher - Registrarse" with a dark background and a pixelated, 8-bit aesthetic. The word "REGISTRO" is centered at the top in a white, pixelated font. Below it, there are several input fields with labels in Spanish: "Nombre:" (with a yellow highlight), "Apellidos:", "Nombre de usuario:", "Contraseña:" (with a small eye icon to its right), "Email:", and "Fecha de nacimiento:". At the bottom of the form are two buttons: "Aceptar" and "Cancelar". The background features faint, pixelated graphics of a character and some architectural elements. In the top right corner, the text "8-Bit" is visible.

Imagen representativa del registro de un usuario.

La siguiente imagen representa el registro de un usuario donde debe completar todos los campos para que quede registrado en la base de datos.

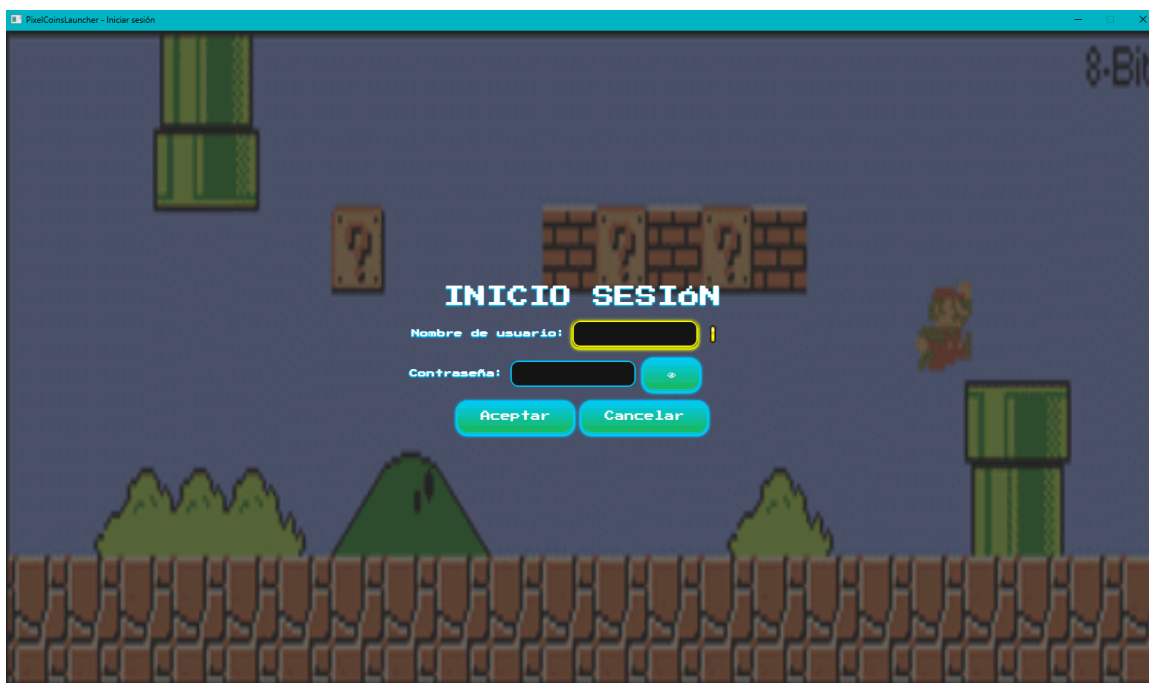


Imagen representativa de inicio sesión del usuario.

Esta imagen representa la ventana del inicio de sesión, donde el usuario debe de registrarse antes si no tiene una cuenta registrada, si tiene cuenta registrada, deberá de completar los campos para pasar a la siguiente escena.

- **com.project.PixelCoinsWindow:** tras iniciar sesión, nos dirige a esta ventana, donde se gestiona la cuenta, la biblioteca de juegos, la tienda y el sistema anti-AFK. Al entrar en esta escena, vemos a la izquierda un menú con tres botones que nos dirigirá a las siguientes secciones de la aplicación:
 - Cuenta: sección de la aplicación para poder visualizar los datos personales de la cuenta registrada en la base de datos como el nombre de usuario, contraseña y el correo electrónico.

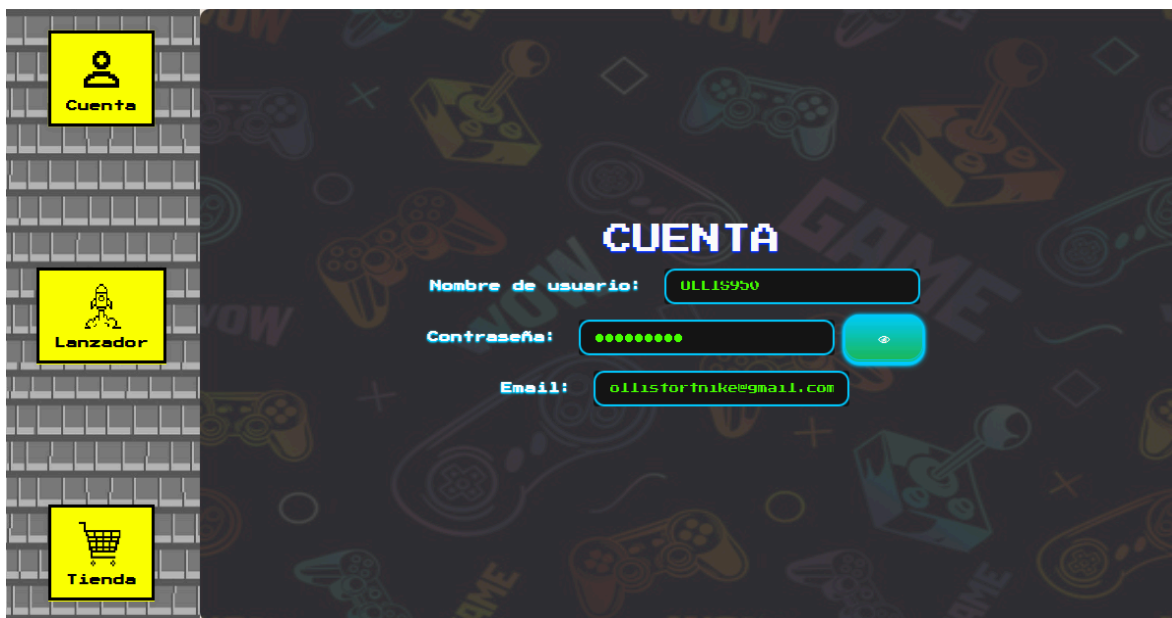


Imagen representativa de la sección de la cuenta del usuario.

- Lanzador: sección de la aplicación que permite al usuario añadir, borrar y ejecutar juego:
 - Añadir juego: si le damos clic en el botón de “Añadir juego” saldrá una nueva ventana para registrar el juego en la biblioteca mediante el nombre de juego, género y la ruta del archivo .exe del juego.

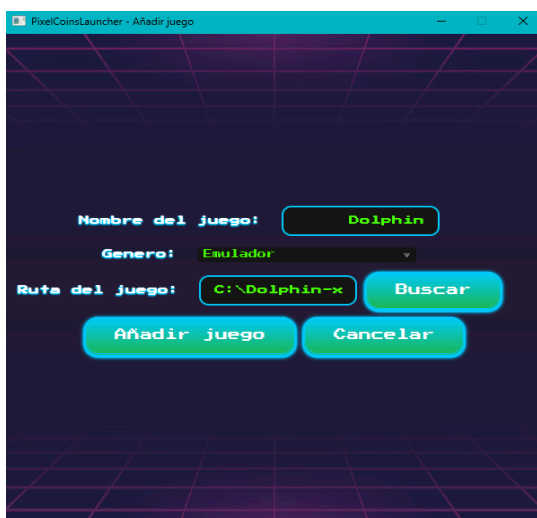


Imagen representativa de la ventana de “Añadir juego”.

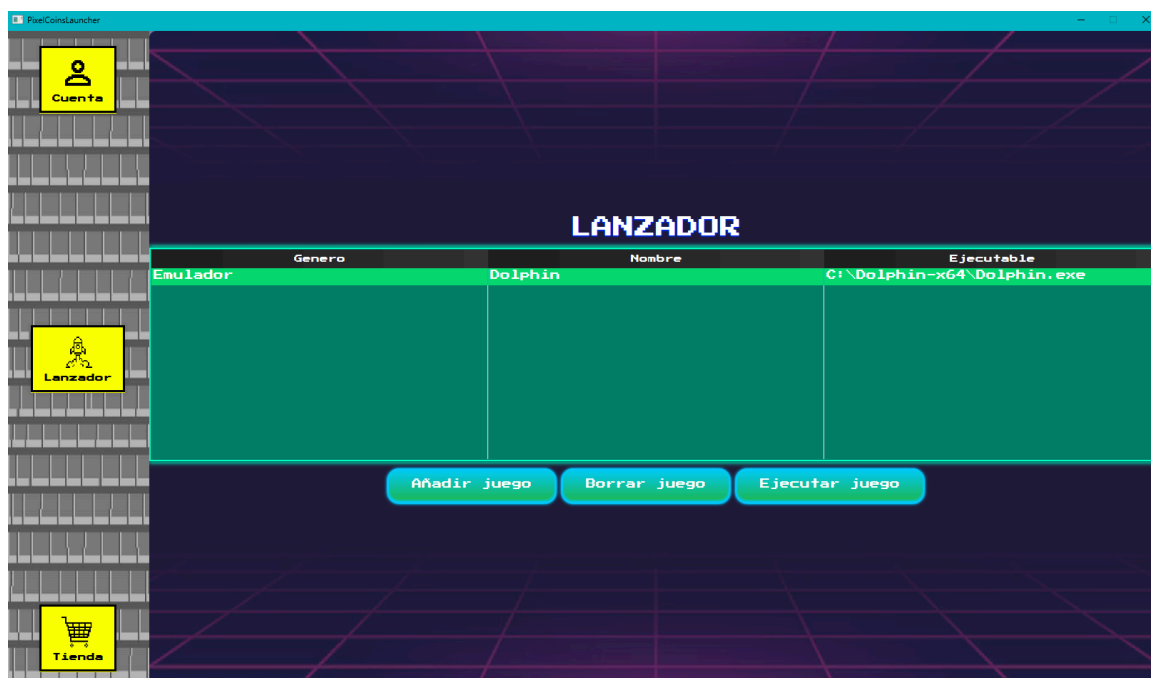


Imagen representativa del juego registrado en la biblioteca de juegos.

- Borrar juego: selecciona un juego que quieras borrar de la biblioteca y desaparecerá de la lista de juegos.

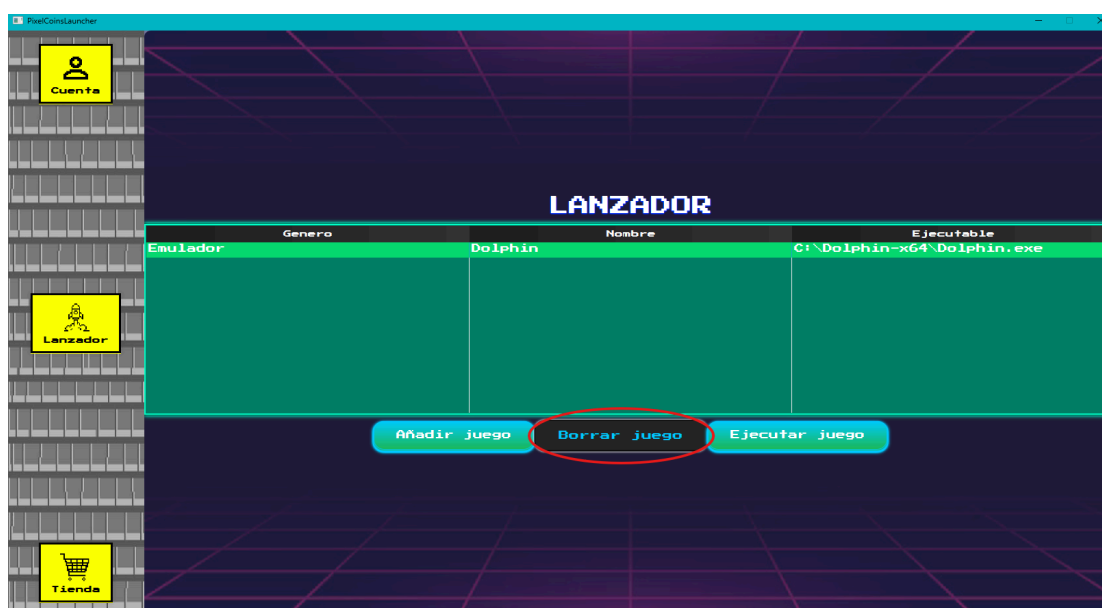


Imagen representativa de la selección del juego para borrar el juego de la biblioteca.

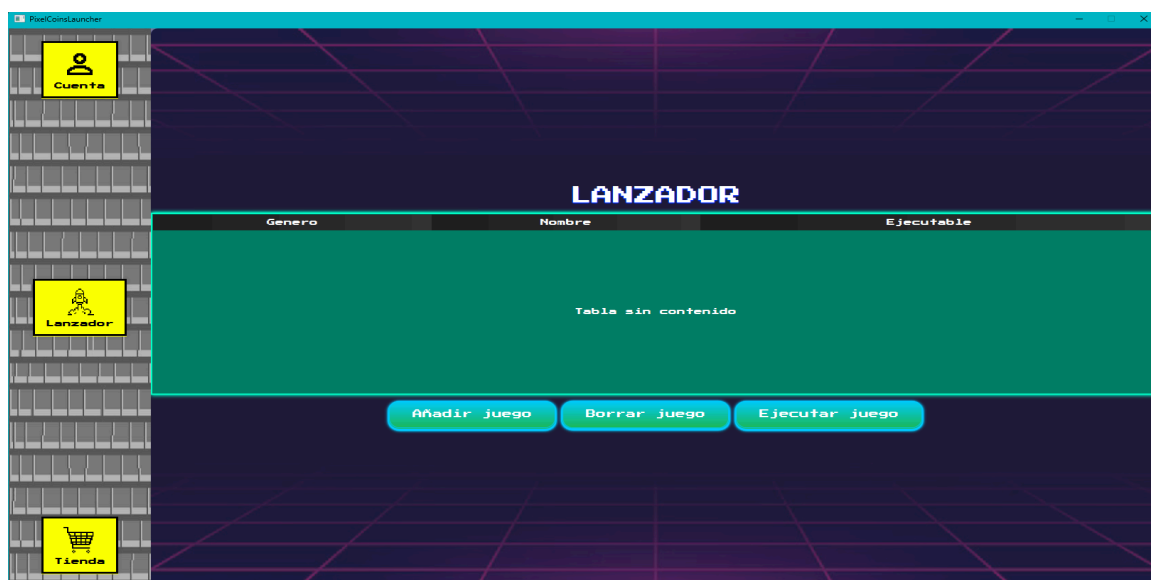


Imagen representativa de la biblioteca de juegos cuando borras el juego.

- Ejecutar juego: selecciona un juego de la biblioteca de juegos y ejecuta el juego desde la aplicación. Es importante mencionar, que al ejecutar el juego, saldrá una nueva ventana donde empezará a monitorizar el tiempo jugado hasta que cerremos el juego o le demos clic a “Detener juego” en la ventana del contador. Cuando queramos salir del juego, saldrá otra ventana que mostrará al usuario el tiempo jugado y las monedas conseguidas.

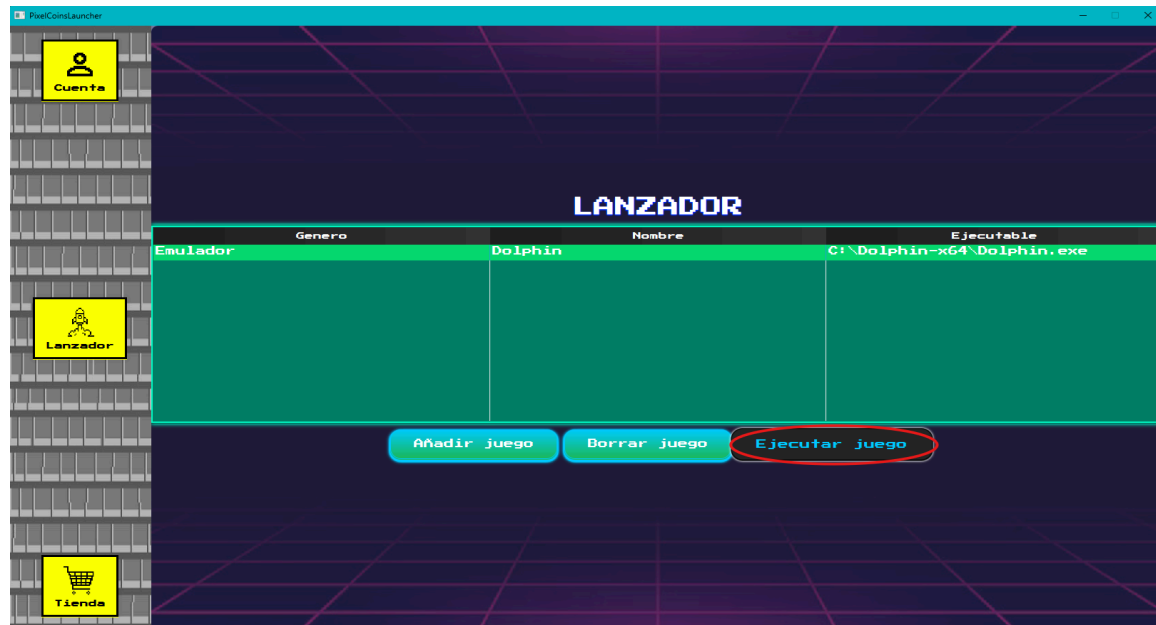


Imagen representativa de la selección del juego para ejecutar el juego.

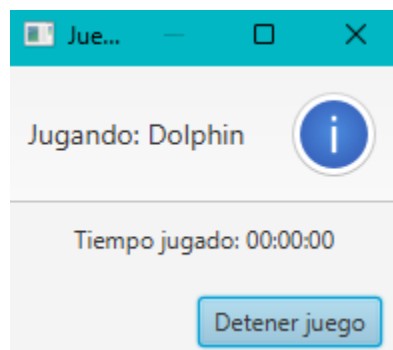


Imagen representativa de la alerta de la monitorización del tiempo jugado.



Imagen representativa de la alerta cuando cerramos el juego con el tiempo jugado y las monedas ganadas.

- Tienda: sección que permitirá al usuario comprar artículos y videojuegos con las monedas ganadas por jugar, también puede observar a la derecha las monedas que tiene el usuario.



Imagen representativa de la sección de la tienda



2. Modelo de datos (Modelo)

El modelo de datos es aquella capa de la arquitectura del Modelo-Vista-Controlador que define las entidades principales del sistema y sus atributos. En este caso, encontramos las siguientes entidades:

- User: representación de un usuario. Sus atributos son: nombre, apellidos, nombre de usuario, contraseña, email, fecha de nacimiento, id del usuario y las monedas del usuario.

```
/**
 * Clase que representa un usuario en el sistema. Contiene información básica
 * sobre el usuario, como su nombre, apellidos, nombre de usuario, contraseña,
 * correo electrónico, fecha de nacimiento, identificador único y saldo de
 * monedas.
 */
You, 2 weeks ago | 1 author (You)
public class User {

    // Atributos
    private String name;
    private String lastName;
    private String userName;
    private String password;
    private String email;
    private LocalDateTime birthday;
    private Long id;
    private int balance;
}
```

Imagen representativa de los atributos de la entidad User.

- Game: representación de un juego. Sus atributos son: id del juego, nombre, género y ruta del archivo .exe.



```
/**
 * Clase que representa un juego en el sistema. Contiene información básica
 * sobre el juego, como su nombre, género y ruta ejecutable.
 */
You, 2 weeks ago | 1 author (You)
public class Game {

    // Atributos
    private Long id;
    private String name;
    private String genre;
    private String exePath;
```

Imagen representativa de los atributos de la entidad Game.

- GameSession: representación de una sesión de juego. Sus atributos son: (id del usuario, id del juego, tiempo jugado, fecha del comienzo de la monitorización del juego, fecha del final de la monitorización del juego y el tiempo total del juego.

```
/**
 * Clase que representa una sesión de juego en el sistema. Contiene información
 * sobre el juego jugado, el usuario que lo jugó, el tiempo de inicio, el tiempo
 * de finalización y la duración total de la sesión.
 */
You, 2 weeks ago | 1 author (You)
public class GameSession {

    // Atributos
    private Long id;
    private Long gameId;
    private Long userId;
    private LocalDateTime startTime;
    private LocalDateTime endTime;
    private Long totalTime;
```

Imagen representativa de los atributos de la entidad GameSession.

Estas entidades son la consecuencia de que encapsulen la información y utilicen la transferencia de datos entre la interfaz gráfica (UI) y la base de datos.



3. Repositorio (Controlador):

El repositorio es aquella capa de la arquitectura del Modelo-Vista-Controlador que se va encargar de gestionar y manipular los datos para reflejar en la vista (UI).

Antes de explicar los repositorios del proyecto, es importante mencionar que para poder controlar los datos, se necesita conectar la base de datos con la programación orientada a objetos. Entonces, existe la clase “ConnectionDB”, en la cual se va a encargar de conectar la base de datos almacenada en Railway gracias al controlador o driver de JDBC mediante un fichero de configuración (config.properties) para asegurar la seguridad de los datos del usuario.

```
public class ConnectionDB {  
  
    private static final Properties props = new Properties();  
  
    static {  
        try (InputStream input = ConnectionDB.class.getClassLoader().getResourceAsStream(name:"com/project/config/config.properties")) {  
            if (input != null) {  
                props.load(input);  
            } else {  
                throw new RuntimeException(message:"No se encontró el archivo config.properties");  
            }  
        } catch (IOException e) {  
            throw new RuntimeException(message:"Error al cargar config.properties", e);  
        }  
    }  
  
    /**  
     * URL de la base de datos. Especifica la ubicación de la base de datos en  
     * Railway MySQL.  
     */  
    private static final String DB_URL = props.getProperty(key:"db.url");  
  
    /**  
     * Nombre de usuario para la conexión a la base de datos. Especifica el  
     * usuario de la base de datos en Railway SQL  
     */  
    private static final String DB_USER = props.getProperty(key:"db.user");  
  
    /**  
     * Contraseña para la conexión a la base de datos. Especifica la contraseña  
     * de la base de datos en Railway SQL  
     */  
    private static final String DB_PASS = props.getProperty(key:"db.pass");  
  
    /**  
     * Obtiene una conexión a la base de datos MySQL.  
     *  
     * @return Un objeto {@link Connection} que representa la conexión a la base  
     * de datos.  
     * @throws SQLException Si ocurre un error al intentar conectarse a la base  
     * de datos.  
     */  
    public static Connection getConnection() throws SQLException {  
        return DriverManager.getConnection(DB_URL, DB_USER, DB_PASS);  
    }  
}
```

Imagen representativa de la clase “ConnectionDB” que conecta la base de datos almacenada en Railway con la POO (Programación Orientada a Objetos)



```
1 db.url=
2 db.user=
3 db.pass=
```

Imagen representativa del fichero de configuración (config.properties) donde se almacenan los parámetros de la conexión de la base de datos.

Ya explicado la conexión de la base de datos, podemos explicar cada repositorio:

- UserRepository: métodos para crear un usuario cuando el usuario se registra en la base de datos y se valida el inicio de sesión haciendo consultas si existe o no el nombre del usuario y el correo electrónico registrado en la base de datos, también se hace una consulta de las monedas para mostrarlas en la tienda.

```
public boolean CreateUser(User user) {
    if (ValidateUserRegisterUserName(user.getUserName())) {
        System.out.println(x:"El usuario ya existe en la base de datos");
        return false;
    }

    if (ValidateUserRegisterEmail(user.getEmail())) {
        System.out.println(x:"El email ya existe en la base de datos");
        return false;
    }

    String query = "INSERT INTO Usuario (name, lastName, userName, password, email, birthday) VALUES (?, ?, ?, ?, ?, ?)";

    try (Connection conn = ConnectionDB.getConnection(); PreparedStatement pstmt = conn.prepareStatement(query, Statement.RETURN_GENERATED_KEYS)) {

        pstmt.setString(parameterIndex:1, user.getName());
        pstmt.setString(parameterIndex:2, user.getLastName());
        pstmt.setString(parameterIndex:3, user.getUserName());
        pstmt.setString(parameterIndex:4, user.getPassword());
        pstmt.setString(parameterIndex:5, user.getEmail());
        pstmt.setObject(parameterIndex:6, user.getBirthday());

        int rows = pstmt.executeUpdate();

        if (rows == 0) {
            return false;
        }

        try (ResultSet generatedKeys = pstmt.getGeneratedKeys()) {
            if (generatedKeys.next()) {
                Long generatedId = generatedKeys.getLong(columnIndex:1);
                user.setId(generatedId);
                return true;
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return false;
}
```

Imagen representativa del método "CreateUser" para crear un usuario cuando se registre en la base de datos.



```
/**
 * Valida si un nombre de usuario ya está registrado en la base de datos.
 *
 * @param userName El nombre de usuario a validar.
 * @return {@code true} si el nombre de usuario ya existe, {@code false} en
 * caso contrario.
 */
public boolean ValidateUserRegisterUserName(String userName) {
    String query = "SELECT 1 FROM Usuario WHERE userName = ? LIMIT 1";

    try (Connection conn = ConnectionDB.getConnection(); PreparedStatement pstmt = conn.prepareStatement(query)) {
        pstmt.setString(parameterIndex:1, userName);

        ResultSet rs = pstmt.executeQuery();
        return rs.next();
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return false;
}

/**
 * Valida si un correo electrónico ya está registrado en la base de datos.
 *
 * @param email El correo electrónico a validar.
 * @return {@code true} si el correo electrónico ya existe, {@code false} en
 * caso contrario.
 */
public boolean ValidateUserRegisterEmail(String email) {
    String query = "SELECT 1 FROM Usuario WHERE email = ? LIMIT 1";

    try (Connection conn = ConnectionDB.getConnection(); PreparedStatement pstmt = conn.prepareStatement(query)) {
        pstmt.setString(parameterIndex:1, email);

        ResultSet rs = pstmt.executeQuery();
        return rs.next();
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return false;
}
```

Imagen representativa de los métodos de validación del nombre de usuario y correo electrónico.

```
public int showCoins(String username) {
    String query = "SELECT balance FROM Usuario Where userName = ?";
    try (Connection conn = ConnectionDB.getConnection(); PreparedStatement pstmt = conn.prepareStatement(query)) {
        pstmt.setString(parameterIndex:1, username);
        ResultSet rs = pstmt.executeQuery();

        if (rs.next()) {
            return rs.getInt(columnLabel:"balance");
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return -1;
}
```

Imagen representativa del método "showCoins" para mostrar las monedas en la tienda.



- GameRepository: métodos para añadir en la base de datos los juegos, eliminar los juegos y listar los juegos por usuarios.

```
public static void addListGames(Game game, Long userId) {
    String query = "INSERT INTO games (name, genre, exePath) VALUES (?, ?, ?)";
    try (Connection conn = ConnectionDB.getConnection(); PreparedStatement pstmt = conn.prepareStatement(query, Statement.RETURN_GENERATED_KEYS)) {
        pstmt.setString(parameterIndex:1, game.getName());
        pstmt.setString(parameterIndex:2, game.getGenre());
        pstmt.setString(parameterIndex:3, game.getExePath());
        pstmt.executeUpdate();

        try (ResultSet rs = pstmt.getGeneratedKeys()) {
            if (rs.next()) {
                game.setId(rs.getLong(columnIndex:1));
            }
        }

        String linkQuery = "INSERT INTO GameSession (game_id, user_id, start_time, end_time, total_time) VALUES (?, ?, NOW(), NOW(), 0)";
        try (PreparedStatement linkStmt = conn.prepareStatement(linkQuery)) {
            linkStmt.setLong(parameterIndex:1, game.getId());
            linkStmt.setLong(parameterIndex:2, userId);
            linkStmt.executeUpdate();
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

Imagen representativa del método “addListGames” para añadir juegos en la biblioteca de juegos.



```
public static void deleteListGames(long userId, long gameId) {
    String deleteRelation = "DELETE FROM Usuario_Games WHERE user_id = ? AND game_id = ?";
    String checkRelation = "SELECT COUNT(*) FROM Usuario_Games WHERE game_id = ?";
    String deleteSessions = "DELETE FROM GameSession WHERE game_id = ?";
    String deleteGame = "DELETE FROM games WHERE id = ?";

    try (Connection conn = ConnectionDB.getConnection()) {
        // 1. Eliminar la relación usuario-juego
        try (PreparedStatement pstmt = conn.prepareStatement(deleteRelation)) {
            pstmt.setLong(parameterIndex:1, userId);
            pstmt.setLong(parameterIndex:2, gameId);
            pstmt.executeUpdate();
        }

        // 2. Verificar si el juego sigue asignado a algún usuario
        try (PreparedStatement pstmt = conn.prepareStatement(checkRelation)) {
            pstmt.setLong(parameterIndex:1, gameId);
            try (ResultSet rs = pstmt.executeQuery()) {
                if (rs.next() && rs.getInt(columnIndex:1) == 0) {
                    // 3. Eliminar sesiones asociadas al juego
                    try (PreparedStatement delSessions = conn.prepareStatement(deleteSessions)) {
                        delSessions.setLong(parameterIndex:1, gameId);
                        delSessions.executeUpdate();
                    }
                    // 4. Si no está asignado a nadie, eliminar el juego
                    try (PreparedStatement delGame = conn.prepareStatement(deleteGame)) {
                        delGame.setLong(parameterIndex:1, gameId);
                        delGame.executeUpdate();
                        System.out.println(x:"Juego eliminado completamente de la base de datos.");
                    }
                } else {
                    System.out.println(x:"Juego eliminado solo de la lista del usuario.");
                }
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

Imagen representativa del método “deleteListGames” para eliminar un juego de la biblioteca de juegos dependiendo de qué cuenta de usuario se está utilizando.



```
public static List<Game> showListGames(Long userId) {
    List<Game> games = new ArrayList<>();
    String query = "SELECT g.id, g.name, g.genre, g.exePath "
        + "FROM games g "
        + "JOIN GameSession gs ON g.id = gs.game_id "
        + "WHERE gs.user_id = ?";

    try (Connection conn = ConnectionDB.getConnection(); PreparedStatement pstmt = conn.prepareStatement(query)) {
        pstmt.setLong(parameterIndex:1, userId);
        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                Game game = new Game();
                game.setId(rs.getLong(columnLabel:"id"));
                game.setName(rs.getString(columnLabel:"name"));
                game.setGenre(rs.getString(columnLabel:"genre"));
                game.setExePath(rs.getString(columnLabel:"exePath"));
                games.add(game);
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return games;
}
```

Imagen representativa del método “showListGames” para mostrar la biblioteca de juegos en el lanzador.

- GameSessionRepository: métodos para guardar las sesiones de juego, consultar el tiempo jugado y las monedas ganadas.

```
public void saveGameSession(Long gameId, Long userId, LocalDateTime startTime, LocalDateTime endTime, Long totalSeconds) {
    String insertQuery = "INSERT INTO GameSession (game_id, user_id, start_time, end_time, total_time) VALUES (?, ?, ?, ?, ?)";
    String updateQuery = "UPDATE Usuario SET balance = balance + ? WHERE id = ?";

    int coinsMin = 10;
    int earnedCoins = (int) (totalSeconds / 60) * coinsMin;

    try (Connection conn = ConnectionDB.getConnection(); PreparedStatement stmtSession = conn.prepareStatement(insertQuery); PreparedStatement stmtBalance = conn.prepareStatement(updateQuery)) {
        stmtSession.setLong(parameterIndex:1, gameId);
        stmtSession.setLong(parameterIndex:2, userId);
        stmtSession.setTimestamp(parameterIndex:3, Timestamp.valueOf(startTime));
        stmtSession.setTimestamp(parameterIndex:4, Timestamp.valueOf(endTime));
        stmtSession.setLong(parameterIndex:5, totalSeconds);
        stmtSession.executeUpdate();

        stmtBalance.setInt(parameterIndex:1, earnedCoins);
        stmtBalance.setLong(parameterIndex:2, userId);
        stmtBalance.executeUpdate();

        System.out.println("Se añadieron " + earnedCoins + " monedas al usuario " + userId);
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

Imagen representativa del método “saveGameSession” para guardar la sesión de juego cuando termina de jugar el usuario.



```
public Long getTotalPlayedTime(Long gameId, Long userId) {
    String query = "SELECT SUM(total_time) AS total FROM GameSession WHERE game_id = ? AND user_id = ?";

    try (Connection conn = ConnectionDB.getConnection(); PreparedStatement pstmt = conn.prepareStatement(query)) {
        pstmt.setLong(parameterIndex:1, gameId);
        pstmt.setLong(parameterIndex:2, userId);
        ResultSet rs = pstmt.executeQuery();

        if (rs.next()) {
            return rs.getLong(columnLabel:"total");
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return 0;
}
```

Imagen representativa del método "getTotalPlayedTime" para mostrar en la alerta el tiempo total que ha jugado el usuario.

```
public int getUserBalance(int userId) {
    String query = "SELECT balance FROM Users WHERE id = ?";
    try (Connection conn = ConnectionDB.getConnection(); PreparedStatement pstmt = conn.prepareStatement(query)) {
        pstmt.setInt(parameterIndex:1, userId);
        ResultSet rs = pstmt.executeQuery();
        if (rs.next()) {
            return rs.getInt(columnLabel:"balance");
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return 0;
}
```

Imagen representativa del método "getUserBalance" para mostrar las monedas conseguidas en la sesión de juego del usuario.

4. Utilidades (Utils):

Esta capa de la arquitectura del sistema se va encargar de utilizar la interacción del sistema operativo para detectar los movimientos del teclado y ratón y la detección de la ventana activa del usuario para poder crear un sistema anti-AFK, es decir, un sistema que permita prohibir o penalizar la inactividad del usuario mientras se registra la sesión de juego. Para ello, se han utilizado librerías como JNativeHook (detección de los movimientos del teclado y ratón a través de la interacción del sistema operativo) y Jna (detección de la ventana activa en la que está posicionado el usuario). Entonces, existen dos clases que permiten esta funcionalidad:



```
public class WindowsChecker {

    /**
     * Obtiene el título de la ventana activa en el sistema operativo Windows.
     *
     * @return Una cadena que representa el título de la ventana activa. Si no
     * hay una ventana activa, devuelve una cadena vacía.
     */
    public static String GetActiveWindowTitle() {
        // Obtener el identificador de la ventana activa
        HWND hwnd = User32.INSTANCE.GetForegroundWindow();

        // Obtener el título de la ventana activa
        char[] windowText = new char[512];
        User32.INSTANCE.GetWindowText(hwnd, windowText, nMaxCount:512);

        return Native.toString(windowText);
    }

    /**
     * Comprueba si una ventana específica está activa.
     *
     * @param windowName El nombre de la ventana que se desea comprobar.
     * @return {@code true} si la ventana especificada está activa,
     * {@code false} en caso contrario.
     */
    public static boolean isWindowActive(String windowName) {
        String activeWindowTitle = GetActiveWindowTitle();
        if (activeWindowTitle == null) {
            return false;
        }
        return activeWindowTitle.equalsIgnoreCase(windowName);
    }

    /**
     * Método principal para probar la funcionalidad de la clase. Muestra el
     * título de la ventana activa en la consola.
     *
     * @param args Argumentos de la línea de comandos (no utilizados).
     */
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        // Llamar al método estático y mostrar el título de la ventana activa
        String activeWindowTitle = WindowsChecker.GetActiveWindowTitle();
        System.out.println("Ventana activa: " + activeWindowTitle);
    }
}
```

Imagen representativa de la clase “WindowsChecker” que permite obtener el título de la ventana activa y detectar si el usuario está en el juego o no.



```
public class GlobalMouseListener implements NativeMouseInputListener {

    private final String targetWindowName; // Nombre de la ventana objetivo
    private boolean isWindowActive;
    private final Runnable onActivityDetected; // Variable global para indicar si la ventana está activa

    /**
     * Constructor que inicializa el listener global del ratón.
     *
     * @param targetWindowName El nombre de la ventana objetivo que se debe
     * monitorear.
     * @param onActivityDetected Una acción que se ejecutará cuando se detecte
     * actividad del ratón.
     * @throws IllegalArgumentException Si el nombre de la ventana objetivo es
     * nulo o está vacío.
     */
    public GlobalMouseListener(String targetWindowName, Runnable onActivityDetected) {
        if (targetWindowName == null || targetWindowName.isEmpty()) {
            throw new IllegalArgumentException(s:"El nombre de la ventana no puede ser nulo o vacío.");
        }
        this.onActivityDetected = onActivityDetected;
        this.targetWindowName = targetWindowName;
        this.isWindowActive = false;

        try {
            // Desactivar los logs de JNativeHook
            Logger logger = Logger.getLogger(GlobalScreen.class.getPackage().getName());
            logger.setLevel(Level.OFF);
            logger.setUseParentHandlers(useParentHandlers:false);

            // Registrar el hook global
            GlobalScreen.registerNativeHook();
            GlobalScreen.getInstance().addNativeMouseListener(this);
            //GlobalScreen.addNativeMouseMotionListener(this);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Imagen representativa de la clase “GlobalMouseListener” que permite utilizar un listener (detector) para capturar los eventos del ratón y verificar si la ventana activa coincide con la ventana del juego.

Para poder verificar que el sistema funciona, existe un método llamado “afkSystem” en “PixelCoinsWindow”, donde se instancia ambas clases y se verifica si el usuario no está activo o no está en la ventana activa del juego y con ello se penaliza al usuario parando el contador y penalizando las monedas conseguidas. Podemos observar que al ejecutar el juego se muestra en la consola todos los movimientos que se realizan con el ratón y se imprimirá si el usuario está en la ventana del juego o está inactivo.



```
/**
 * Sistema anti-AFK que pausa el juego si no hay actividad o si la ventana
 * no está activa.
 *
 * @param timeline El temporizador del juego.
 */
public void afkSystem(Timeline timeline) {
    // Bandera para controlar si el temporizador ya está en pausa
    boolean[] isGamePaused = {false};

    if (windowTitle == null || windowTitle.isEmpty()) {
        throw new IllegalArgumentException(s:"El título de la ventana no puede ser nulo o vacío.");
    }

    antiAfkTimeline = new Timeline(new KeyFrame(javafx.util.Duration.seconds(s:1), event -> {
        boolean isWindowActive = WindowsChecker.isWindowActive(windowTitle);
        Long currentTime = System.currentTimeMillis();
        Long inactivityTime = currentTime - lastActivityTime.get();

        // Verificamos si la ventana no está activa o si ha pasado el tiempo de inactividad
        if (!isWindowActive || inactivityTime > 60000) {
            if (timeline.getStatus() == Timeline.Status.RUNNING && !isGamePaused[0]) {
                // Pausamos el juego solo si no está en pausa
                timeline.pause();
                isGamePaused[0] = true; // Actualizamos el estado a "pausado"
                System.out.println(x:"[Anti-AFK]: Juego pausado por inactividad o ventana no activa.");
            }
        } else {
            if (timeline.getStatus() == Timeline.Status.PAUSED && isGamePaused[0]) {
                // Reanudamos el juego solo si estaba en pausa
                timeline.play();
                isGamePaused[0] = false; // Actualizamos el estado a "jugando"
                System.out.println(x:"[Anti-AFK]: Juego reanudado.");
            }
        }
    }));

    antiAfkTimeline.setCycleCount(Timeline.INDEFINITE);
    antiAfkTimeline.play();

    // Listener de mouse para actualizar el tiempo de actividad
    mouseListener = new GlobalMouseListener(windowTitle, () -> {
        lastActivityTime.set(System.currentTimeMillis());
    });
}
```

Imagen representativa del método “afkSystem” en “PixelCoinsWindow”.

```
[Anti-AFK]: Juego pausado por inactividad o ventana no activa.
```

Imagen representativa cuando el software detecta que el usuario está inactivo o no está en la ventana activa del juego mostrado en consola.



```
LowLevelMouseProc(): Motion Notified. (880, 592)
LowLevelMouseProc(): Motion Notified. (880, 594)
LowLevelMouseProc(): Motion Notified. (880, 595)
LowLevelMouseProc(): Motion Notified. (880, 593)
LowLevelMouseProc(): Motion Notified. (880, 592)
LowLevelMouseProc(): Motion Notified. (880, 590)
LowLevelMouseProc(): Motion Notified. (880, 588)
LowLevelMouseProc(): Motion Notified. (881, 587)
LowLevelMouseProc(): Motion Notified. (883, 587)
LowLevelMouseProc(): Motion Notified. (884, 587)
LowLevelMouseProc(): Motion Notified. (884, 589)
LowLevelMouseProc(): Motion Notified. (883, 589)
LowLevelMouseProc(): Motion Notified. (881, 589)
LowLevelMouseProc(): Motion Notified. (880, 589)
```

Imagen representativa de la detección de los movimientos del ratón mostrado en consola.

3.4 CODIFICACIÓN

La implementación de PixelCoinsLauncher se ha realizado principalmente en Java, utilizando el framework JavaFX para la interfaz gráfica y JDBC para la persistencia de datos en una base de datos MySQL. La gestión del proyecto y las dependencias se realiza con Maven.

Lenguajes y herramientas utilizadas:

- Lenguaje principal: Java
- Framework de UI: JavaFX
- Persistencia: JDBC (Java Database Connectivity)
- Base de datos: MySQL
- Gestor de dependencias y build: Maven
- Pruebas: JUnit y TestFX
- IDE (Entorno de desarrollo): Visual Studio Code



Estructura del código:

El código está organizado en paquetes siguiendo el patrón MVC:

- com.project: Ventanas principales y lógica de interacción.
- com.project.model: Modelos de datos (User, Game, GameSession).
- com.project.repository: Acceso a datos y lógica de persistencia.
- com.project.utils: Utilidades (por ejemplo, anti-AFK).
- resources/com/project/: Imágenes y estilos CSS.

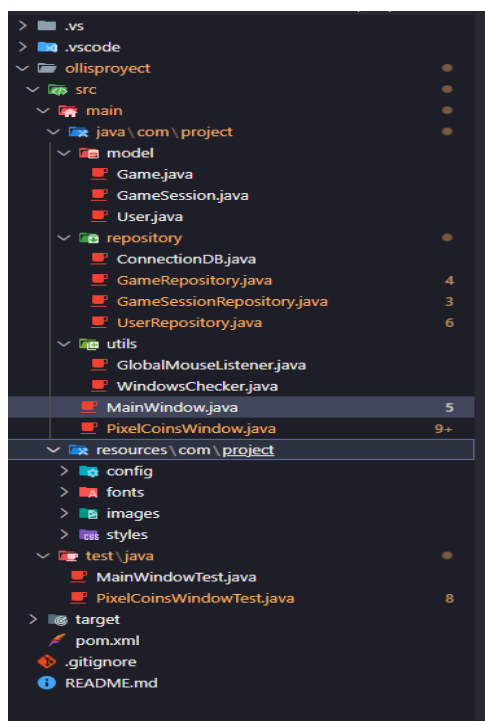


Imagen representativa de la estructura del proyecto



3.5 EJECUCIÓN DE PRUEBAS

Para asegurar que el software funciona de manera correcta, se ha realizado una serie de pruebas mediante las siguientes clases:

- MainWindowTest:
 - testSetAndGetCurrentUser: prueba unitaria para verificar si se establece correctamente el usuario logueado.

```
@Test
void testSetAndGetCurrentUser() {
    mainWindow = new MainWindow();
    User user = new User(name:"Iker", lastName:"Magro", userName:"OLLIS950", password:"password", email:"iker123@example.com", LocalDateTime.now());

    mainWindow.setCurrentUser(user);
    assertEquals(user, mainWindow.getCurrentUser());
}
```

Imagen representativa del método “testSetAndGetCurrentUser”, en la cual se crea un User con sus respectivos atributos y se verifica mediante assertEquals (afirmar si el objeto User creado de prueba es igual al objeto original).

- Pruebas de la UI: pruebas unitarias realizadas con la librería testFX para verificar el funcionamiento correcto de la interacción con la interfaz gráfica. Existen varias pruebas:
 - testRegisterWindow: prueba de interfaz gráfica para verificar el flujo de registro de usuarios. Simula la interacción del usuario con los campos de texto, el selector de fecha y el botón de registro. Verifica que se muestra una alerta de éxito al completar el registro.

```
@Test
void testRegisterWindow() {
    clickOn(query:"Registrarse");
    verifyThat(nodeQuery:"#lblRegister", hasText(text:"REGISTRO"));

    clickOn(query:"#txtNameRegister").write(text:"Iker");
    clickOn(query:"#txtLastNameRegister").write(text:"Magro");
    clickOn(query:"#txtUserNameRegister").write(text:"OLLIS950");
    clickOn(query:"#txtPasswordRegister").write(text:"password");
    clickOn(query:"#txtEmailRegister").write(text:"iker123@example.com");

    interact(() -> {
        DatePicker datePicker = lookup(query:"#datePickerRegister").query();
        datePicker.setValue(java.time.LocalDate.of(year:2000, month:1, dayOfMonth:1));
    });

    verifyThat(nodeQuery:"#datePickerRegister", datePicker
        -> ((DatePicker) datePicker).getValue().toString().equals(anObject:"2000-01-01")
    );

    clickOn(query:"#btnAcceptRegister");
    interact(() -> {
        Alert alert = mainWindow.showAlert(Alert.AlertType.INFORMATION, title:"Información", headerText:null, contentText:"Registro exitoso");
        assertEquals(expected:"Registro exitoso", alert.getContentText());
    });
}
```

Imagen representativa del método “testRegisterWindow”.



- **testLoginWindow:** prueba unitaria para verificar el flujo de inicio de sesión. Simula la interacción del usuario con los campos de texto y el botón de inicio de sesión. Verifica que se muestra una alerta de éxito al iniciar sesión correctamente.

```
@Test
void testLoginWindow() {
    clickOn(query:"Iniciar sesión");

    verifyThat(nodeQuery:"#lblLogin", hasText(text:"INICIO SESIÓN"));

    clickOn(query:"#txtUserNameLogin").write(text:"OLLIS950");
    clickOn(query:"#txtPasswordLogin").write(text:"password123");

    clickOn(query:"#btnAcceptLogin");
    interact() -> {
        Alert alert = mainWindow.showAlert(Alert.AlertType.INFORMATION, title:"Información", headerText:null, contentText:"Inicio de sesión exitoso");
        assertEquals(expected:"Inicio de sesión exitoso", alert.getContentText());
    };
}
```

Imagen representativa del método “testLoginWindow”.

- **testShowDisguisePassword:** prueba unitaria para verificar la funcionalidad de mostrar y ocultar contraseñas. Simula la interacción del usuario con los campos de contraseña y texto visible. Verifica que los valores se transfieren correctamente entre los campos y que la visibilidad cambia.



```
@Test
void testShowDisguisePassword() {
    PasswordField passwordField = new PasswordField();
    TextField textField = new TextField();

    passwordField.setText(value:"password123");
    textField.setText(value:"password123");

    // Alternar a mostrar la contraseña
    mainWindow.showDisguisePassword(passwordField, textField);
    assertEquals(expected:"password123", textField.getText());
    assertEquals(expected:false, passwordField.isVisible());
    assertEquals(expected:true, textField.isVisible());

    // Alternar a ocultar la contraseña
    mainWindow.showDisguisePassword(passwordField, textField);
    assertEquals(expected:"password123", passwordField.getText());
    assertEquals(expected:true, passwordField.isVisible());
    assertEquals(expected:false, textField.isVisible());
}
```

Imagen representativa del método “testShowDisguisePassword”.

- testShowAlert: prueba unitaria para verificar la funcionalidad de mostrar alertas. Crea una alerta de prueba y verifica que los valores de tipo, título y contenido sean correctos.

```
@Test
void testShowAlert() {
    Platform.runLater(() -> {
        Alert alert = mainWindow.showAlert(Alert.AlertType.INFORMATION, title:"Información", headerText:null, contentText:"Mensaje de prueba");
        assertEquals(Alert.AlertType.INFORMATION, alert.getAlertType());
        assertEquals(expected:"Información", alert.getTitle());
        assertEquals(expected:"Mensaje de prueba", alert.getContentText());
    });
}
```

Imagen representativa del método “testShowAlert”.

- PixelCoinsWindowTest:
 - Pruebas de la UI:
 - testShowLaunchScene: prueba unitaria para verificar que la escena del lanzador de juegos se muestra correctamente. Verifica que los elementos de la interfaz, como la tabla de juegos y los botones, están presentes.



```
@Test
void testShowLaunchScene() throws Exception {
    FxToolkit.setupStage(stage -> {
        PixelCoinsWindow pixelCoinsWindow = new PixelCoinsWindow(new MainWindow());
        pixelCoinsWindow.start(stage);
        pixelCoinsWindow.showLaunchScene();
    });

    verifyThat(nodeQuery:".table-view", (TableView<Game> table) -> table.getColumns().size() == 3);

    verifyThat(nodeQuery:"#btnAddGame", (Button btn) -> btn.getText().equals(anObject:"Añadir juego"));
    verifyThat(nodeQuery:"#btnDeleteGame", (Button btn) -> btn.getText().equals(anObject:"Borrar juego"));
    verifyThat(nodeQuery:"#btnExecuteGame", (Button btn) -> btn.getText().equals(anObject:"Ejecutar juego"));
}
```

Imagen representativa del método “testShowLaunchScene”.

- testShowShopScene: prueba unitaria para verificar que la escena de la tienda se muestra correctamente. Verifica que los elementos de la interfaz, como el saldo de monedas y los mensajes, están presentes.

```
@Test
void testShowShopScene() throws Exception {
    FxToolkit.setupStage(stage -> {
        MainWindow mainWindow = new MainWindow();
        mainWindow.setCurrentUser(new User(userName:"OLLIS950", password:"password123", email:"iker123@example.com"));
        PixelCoinsWindow pixelCoinsWindow = new PixelCoinsWindow(mainWindow);
        pixelCoinsWindow.start(stage);
        pixelCoinsWindow.showShopScene();
    });

    verifyThat(nodeQuery:"#lblBalance", NodeMatchers.isNotNull());
    verifyThat(nodeQuery:"#lblComingSoon", (Label lbl) -> lbl.getText().contains(s:"Proximamente productos en la tienda..."));
}
```

Imagen representativa del método “testShowShopScene”.

- testShowAccountScene: prueba unitaria para verificar que la escena de la cuenta del usuario se muestra correctamente. Verifica que los campos de texto contienen la información del usuario autenticado.



```
@Test
void testShowAccountScene() throws Exception {
    FxToolkit.setupStage(stage -> {
        MainWindow mainWindow = new MainWindow();
        mainWindow.setCurrentUser(new User(userName:"OLLIS950", password:"password123", email:"iker123@example.com"));
        pixelCoinsWindow = new PixelCoinsWindow(mainWindow);
        pixelCoinsWindow.start(stage);
        pixelCoinsWindow.showAccountScene();
    });

    verifyThat(nodeQuery:"#txtUserNameAccount", (TextField txt) -> txt.getText().equals(anObject:"OLLIS950"));
    verifyThat(nodeQuery:"#txtPasswordAccount", (PasswordField txt) -> txt.getText().equals(anObject:"password123"));
    verifyThat(nodeQuery:"#txtEmailAccount", (TextField txt) -> txt.getText().equals(anObject:"iker123@example.com"));
}
```

Imagen representativa del método “testShowAccountScene”.

- testRegisterGamesWindow_Success: prueba unitaria para verificar que un juego se registra correctamente. Simula la interacción del usuario con los campos de texto y botones en la ventana de registro de juegos.

```
@Test
void testRegisterGamesWindow_Success() throws Exception {
    FxToolkit.setupStage(stage -> {
        PixelCoinsWindow pixelCoinsWindow = new PixelCoinsWindow(new MainWindow());
        pixelCoinsWindow.start(stage);

        Platform.runLater(pixelCoinsWindow::RegisterGamesWindow);
    });

    clickOn(query:"#txtNameGame").write(text:"Nuevo Juego");
    clickOn(query:"#comboBox").clickOn(query:"Aventura");
    clickOn(query:"#txtPath").write(text:"C:\\Dolphin-x64\\Dolphin.exe");

    clickOn(query:"#btnAddGameRegisterGame");
    interact(() -> {
        Alert alert = pixelCoinsWindow.showAlert(Alert.AlertType.INFORMATION, title:"Éxito", headerText:null, contentText:"Juego agregado correctamente");
        assertEquals(expected:"Juego agregado correctamente", alert.getContentText());
    });
}
```

Imagen representativa del método “testRegisterGamesWindow_Success”.



- testRegisterGamesWindow_MissingFields: prueba unitaria para verificar que se muestra un error si faltan campos obligatorios al intentar registrar un juego.

```
@Test
void testRegisterGamesWindow_MissingFields() throws Exception {
    FxToolkit.setupStage(stage -> {
        PixelCoinsWindow pixelCoinsWindow = new PixelCoinsWindow(new MainWindow());
        pixelCoinsWindow.start(stage);

        // Abrir la ventana de registro de juegos
        Platform.runLater(pixelCoinsWindow::RegisterGamesWindow);
    });

    clickOn(query: "#btnAddGameRegisterGame");

    interact(() -> {
        Alert alert = pixelCoinsWindow.showAlert(Alert.AlertType.ERROR, title: "Error", headerText: null, contentText: "Todos los campos deben completarse");
        assertEquals(expected: "Todos los campos deben completarse", alert.getContentText());
    });
}
```

Imagen representativa del método "testRegisterGamesWindow_MissingFields".

- testRegisterGamesWindow_GameAlreadyExists: prueba unitaria para verificar que se muestra un error si el juego ya existe en la lista al intentar registrarlo nuevamente.

```
@Test
void testRegisterGamesWindow_GameAlreadyExists() throws Exception {
    FxToolkit.setupStage(stage -> {
        PixelCoinsWindow pixelCoinsWindow = new PixelCoinsWindow(new MainWindow());
        pixelCoinsWindow.start(stage);

        Platform.runLater(pixelCoinsWindow::RegisterGamesWindow);
    });

    clickOn(query: "#txtNameGame").write(text: "Juego Existente");
    clickOn(query: "#comboBox").clickOn(query: "Aventura");
    clickOn(query: "#txtPath").write(text: "C:\\Dolphin-x64\\Dolphin.exe");

    clickOn(query: "#btnAddGameRegisterGame");

    interact(() -> {
        Alert alert = pixelCoinsWindow.showAlert(Alert.AlertType.ERROR, title: "Error", headerText: null, contentText: "El juego ya está en la lista");
        assertEquals(expected: "El juego ya está en la lista", alert.getContentText());
    });
}
```

Imagen representativa "testRegisterGamesWindow_GameAlreadyExists".



- testExecuteGame_NoGameSelected: prueba unitaria para verificar que se muestra un error si no se selecciona un juego antes de intentar ejecutarlo.

```
@Test
void testExecuteGame_NoGameSelected() throws Exception {
    FxToolkit.setupStage(stage -> {
        PixelCoinsWindow pixelCoinsWindow = new PixelCoinsWindow(new MainWindow());
        pixelCoinsWindow.start(stage);

        pixelCoinsWindow.mainWindow.setCurrentUser(new User(userName:"OLLIS950", password:"password123", email:"iker@example.com"));
        TableView<Game> table = new TableView<>();
        Platform.runLater(() -> pixelCoinsWindow.executeGame(table));
    });

    interact(() -> {
        Alert alert = pixelCoinsWindow.showAlert(Alert.AlertType.ERROR, title:"Error", headerText:null, contentText:"Selecciona un juego antes de ejecutar");
        assertEquals(expected:"Selecciona un juego antes de ejecutar", alert.getContentText());
    });
}
```

Imagen representativa “testExecuteGame_NoGameSelected”.

- testExecuteGame_ErrorExecutingGame: Prueba unitaria para verificar que se muestra un error si ocurre un problema al intentar ejecutar un juego.

```
@Test
void testExecuteGame_ErrorExecutingGame() throws Exception {
    FxToolkit.setupStage(stage -> {
        PixelCoinsWindow pixelCoinsWindow = new PixelCoinsWindow(new MainWindow());
        pixelCoinsWindow.start(stage);

        pixelCoinsWindow.mainWindow.setCurrentUser(new User(userName:"OLLIS950", password:"password123", email:"iker@example.com"));

        TableView<Game> table = new TableView<>();
        Game game = new Game(name:"Invalid Game", genre:"Aventura", exePath:"C:\\invalid\\path.exe");
        table.getItems().add(game);
        table.getSelectionModel().select(game);

        Platform.runLater(() -> pixelCoinsWindow.executeGame(table));
    });

    interact(() -> {
        Alert alert = pixelCoinsWindow.showAlert(Alert.AlertType.ERROR, title:"Error", headerText:null, contentText:"No se pudo ejecutar el juego.");
        assertEquals(expected:"No se pudo ejecutar el juego.", alert.getContentText());
    });
}
```

Imagen representativa del método “testExecuteGame_ErrorExecutingGame”.



3.6 VERIFICACIÓN

La verificación del sistema PixelCoinsLauncher se ha realizado mediante pruebas unitarias, de integración y validación manual, asegurando que cada requisito funcional esté correctamente implementado y funcionando según lo esperado. Para ello, se han utilizado los tests ubicados en MainWindowTest y PixelCoinsWindowTest.

A continuación, se presenta una tabla cruzada que relaciona cada requisito funcional con el método de verificación utilizado y el resultado obtenido:

Requisito	Prueba unitaria	Resultado
Registro de usuarios	testRegisterWindow	OK
Inicio de sesión	testLoginWindow	OK
Mostrar / Ocultar contraseña	testShowDisguisePassword	OK
Gestión de biblioteca de juegos (añadir, eliminar)	testRegisterGamesWindow_Success, testRegisterGamesWindow_MissingFields, testRegisterGamesWindow_GameAlreadyExists	OK
Ejecución de juegos	testExecuteGame_NoGameSelected, testExecuteGame_ErrorExecutingGame	OK
Visualización de cuenta y monedas	testShowAccountScene, testShowShopScene	OK
Alertas y mensajes de error	testShowAlert	OK

Leyenda:

- OK: Requisito verificado y funcional.
- Las pruebas unitarias y automatizadas están implementadas con JUnit 5 y TestFX.



3.7 MANTENIMIENTO

El software ha sido diseñado pensando en la escalabilidad y la modularidad, lo que facilita su mantenimiento y futuras ampliaciones. A continuación se detallan los aspectos clave:

Escalabilidad:

- La arquitectura en capas y el uso del patrón MVC permiten añadir nuevas funcionalidades, por ejemplo: logros, estadísticas, etc, sin afectar al núcleo del sistema.

Modularidad:

- El código está organizado en paquetes (model, repository, ventanas principales), lo que facilita la localización y modificación de componentes específicos.

Futuras versiones y ampliaciones:

- Se pueden añadir nuevas escenas o ventanas (por ejemplo, para logros, configuración avanzada, soporte multilinguaje).
- Es posible loguearse con autenticaciones externas (Google) o crear sistemas de pago para la tienda.
- El sistema anti-AFK puede evolucionar para detectar más tipos de inactividad o integrarse con otros sistemas operativos.
- La tienda puede ampliarse para permitir compras reales.

Mantenimiento general:

- El uso de Maven facilita la actualización de dependencias y la integración continua.
- El código está documentado y las clases principales tienen comentarios explicativos.



4. TECNOLOGÍAS

Lenguajes de programación:

- Java: lenguaje de programación principal para desarrollar el Backend de la aplicación (métodos, funcionalidades, eventos. conexión base de datos...).
- CSS: lenguaje que va permitir diseñar y poner estilos a todos los componentes del software para que reluzca más atractivo y moderno.
- MySQL: motor de base de datos principal para crear y manipular los datos de la base de datos.

Frameworks y librerías:

- JavaFX (UI): framework de Java que permite crear y diseñar interfaces a nuestro gusto.
- JUnit: librería que va permitir crear pruebas unitarias para poder verificar todas las funcionalidades del proyecto.
- JNativeHook: librería que va permitir detectar los inputs (movimientos) del teclado y ratón para poder detectar si ha habido actividad del usuario o no.
- Jna: librería que va permitir gracias a la interacción del sistema operativo detectar la ventana activa del juego.
- TextFX: librería que va permitir realizar pruebas de UI de JavaFx para verificar si todos los componentes funcionan de manera correcta.
- MySQL-Connector: librería o driver que va permitir conectar la base de datos con la programación orientada a objetos para inyectar y lanzar queries a la base de datos desde Java.

Herramientas:

- Visual Studio Code: entorno de desarrollo que va permitir desarrollar el proyecto de JavaFX.
- Launch4J: herramienta que va permitir transformar el empaquetado del proyecto (archivo .jar) a un ejecutable (.exe).
- Railway: servicio web que va permitir alojar y actuar como host de la base de datos en la nube.
- DBeaver: entorno que va permitir conectar la base de datos con Railway.
- Git: control de versiones que va permitir subir todos los cambios a nivel local.
- GitHub: repositorio que permite subir todos los cambios locales de manera remota.



5. PLANIFICACIÓN DEL PROYECTO

Semana	Actividad	Horas
1	Investigación y desarrollo de la idea del proyecto	8
2	Diseño de arquitectura y base de datos	8
3	Registro de usuarios y validaciones	10
4	Inicio de sesión y validaciones	10
5	Menú (cuenta, lanzador, tienda)	10
6	Desarrollo de la sección de Cuenta	10
7	Desarrollo de biblioteca de juegos (tabla y botones)	10
8	Desarrollo de eventos de los botones de la sección de Lanzador (añadir, borrar y ejecutar juego)	10
9	Ejecución de juegos y registro de sesiones de juego	10
10	Sistema de monedas	8
11	Desarrollo de la sección de la tienda	8
12	Desarrollo de sistema anti-AFK (JNativeHook, Jna y método "afkSystem")	12
13	Pruebas unitarias (JUnit) y de UI (testFX)	10
14	Hojas de estilo (CSS)	8
15	Mejoras, arreglos de la base de datos y empaquetado	8
16	Documentación	8



6. DESARROLLO Y EJECUCIÓN DEL PROYECTO

1. Análisis y definición de requisito:

El proyecto comenzó con identificar una necesidad al usuario: crear una aplicación de escritorio que permitiera gestionar una biblioteca de juegos, registrar sesiones y recompensar el tiempo jugado con monedas virtuales. Se elaboró un listado de requisitos funcionales y no funcionales, así como los principales casos de uso, asegurando una solución.

2. Diseño de la arquitectura y la base de datos:

Se optó por una arquitectura MVC (Modelo-Vista-Controlador) para garantizar la separación de la lógica de negocio y facilitar el mantenimiento. Se diseñó la estructura de paquetes y la base de datos, definiendo las tablas para usuarios, juegos y sesiones de juego. Se eligió JavaFX para la interfaz gráfica por su potencia y compatibilidad con Java.

3. Implementación modular:

El desarrollo se realizó por módulos:

- Ventana principal y autenticación: Se implementó primero la pantalla de bienvenida, registro e inicio de sesión, con validaciones y mensajes de error claros.
- Gestión de usuarios y persistencia: Se crearon los modelos y repositorios para usuarios, asegurando la conexión y operaciones básicas con la base de datos.
- Gestión de juegos: Se desarrolló la funcionalidad para añadir, eliminar y listar juegos, así como la selección y ejecución de los mismos.
- Registro de sesiones y sistema de monedas: Se implementó el registro automático de sesiones de juego y el cálculo de monedas virtuales en función del tiempo jugado.
- Tienda y sistema anti-AFK: Se añadió la tienda (con productos "próximamente") y el sistema anti-AFK, que supuso un reto técnico por la necesidad de detectar actividad real del usuario.

4. Integración de la interfaz gráfica:

Se diseñaron y aplicaron estilos CSS personalizados, imágenes y fondos animados para mejorar la experiencia visual. Se cuidó la usabilidad y la disposición de los elementos en cada ventana.



5. Pruebas y validación:

Se desarrollaron pruebas unitarias y de interfaz con JUnit y TestFX, cubriendo los principales flujos de usuario y validaciones. Además, se realizaron pruebas manuales para asegurar la robustez y la experiencia de usuario.

6. Empaquetado y documentación:

Se utilizó Maven para la gestión del proyecto y la generación del ejecutable. Se preparó la documentación técnica y el README, detallando la estructura, requisitos y modo de uso.

7. Toma de decisiones:

Decisiones clave:

- Uso de JavaFX y MVC: Para garantizar una arquitectura limpia y una interfaz moderna.
- Persistencia con JDBC: Por su sencillez y control directo sobre las operaciones SQL.
- Sistema anti-AFK: Se decidió implementar un control de actividad real para evitar abusos en la obtención de monedas.

Dificultades superadas:

- Gestión de rutas y procesos: ejecutar juegos externos y controlar su ventana supuso retos de compatibilidad y sincronización.
- Validaciones y mensajes de error: se dedicó tiempo a asegurar que los mensajes fueran claros y útiles para el usuario.
- Pruebas de interfaz: automatizar pruebas en JavaFX requirió investigar y adaptar el uso de TestFX.



7. CONCLUSIONES

El desarrollo de este software me ha permitido desarrollar y afianzar conocimientos sobre JavaFx de las que antes no conocía, por ejemplo el uso de CSS para aplicar estilos a los componentes de mi aplicación, también es importante mencionar que he aprendido el uso de herramientas como Maven para poder trabajar de manera más eficiente el proyecto mediante los módulos y dependencias del proyecto y he aprendido a preparar un despliegue real de una aplicación de escritorio con JavaFX gracias a herramientas como Maven y Launch4j.

Por otra parte, me he dado cuenta de que es muy importante utilizar los controles de versiones, porque te ayuda a volver a una versión anterior si falla algo del proyecto.

Este proyecto me ha incentivado a explorar nuevas funcionalidades que podría implementar en un futuro cercano y lejano como implementar la misma aplicación en otras plataformas como móvil y web, a parte de seguir mejorando la interfaz gráfica y otras funcionalidades mínimas como la implementación de poner la aplicación en segundo plano cuando minimice las ventanas para que el usuario no tenga las ventanas abiertas, otra escena donde haya logros por conseguir para aumentar las recompensas, poder loguearse con la autenticación de Google, sistemas de pago para comprar artículos reales, etc.

8. LÍNEAS DE INVESTIGACIÓN Y DESARROLLO FUTURAS

Este sistema puede ofrecer nuevas funcionalidades y mejoras futuras:

- Versión web y móvil: adaptar la aplicación en otras plataformas como en web y en móvil.
- Mejoras en la tienda: implementar un sistema real de compras de artículos, videojuegos y sistemas de pago.
- Sistema de logros: añadir logros para incentivar el uso de la aplicación y mejorar la satisfacción del usuario.

Estas mejoras y funcionalidades permitirán al software seguir evolucionando a una aplicación más completa y que cubra mucho más las necesidades de los usuarios.



9. BIBLIOGRAFÍA

<https://www.youtube.com/watch?v=7VL4kX02FzI> - Configuración y manipulación de DBeaver para vincularlo con Railway (almacenamiento de base de datos MySQL en la nube)

<https://chatgpt.com/> - Apoyo para el diseño de la UI y hojas de estilo y solucionar fallos relacionados con la base de datos.

<https://openjfx.io/openjfx-docs/> - Comandos para la generación del Jar y solucionar fallos de la ejecución del fichero .jar.

<https://railway.com/> - Almacenamiento de la base de datos en la nube.

<https://java-native-access.github.io/jna/4.2.0/> - Documentación de la librería Jna.

<https://javadoc.io/doc/com.1stleg/jnativehook/latest/index.html> - Documentación de la librería JNativeHook.

10. EXTRAS

10.1 ANÁLISIS DE MERCADO

Los usuarios que utilizarían este software sería aquellas personas que quieren llevar un registro de tiempo jugado de una sesión de juego en cada juego y jugadores que les gustaría ganar recompensas por solo jugar y tener un incentivo para jugar mientras ganas monedas para gastarlas en artículos más baratos que en las tiendas convencionales.

Las necesidades que resuelven son:

- Biblioteca de juegos: añadir, eliminar y ejecutar juegos desde una interfaz.
- Registro del tiempo jugado: registra de manera automática las sesiones de juego.
- Sistema de recompensas: incentiva a los jugadores jugar de manera activa a través de conseguir monedas según el tiempo jugado.
- Sistema anti-AFK: asegura recompensas si el usuario se mantiene activo durante la sesión de juego, sino se penalizará al jugador paralizando el tiempo jugado y no recibir monedas.
- Tienda (futura mejora): permitir gastar las monedas en productos o artículos de videojuegos dentro del mismo software.
- Gestión de cuentas: facilitar al usuario el registro, inicio de sesión y administración de los datos del usuario.



10.2 SEGURIDAD Y PROTECCIÓN DE DATOS

Las medidas de seguridad aplicadas en el software incluyen:

1. Validación de entradas del usuario:

- En los formularios de registro e inicio de sesión, se valida que los campos no estén vacíos y que cumplan con formatos específicos (por ejemplo, email válido, contraseña mínima de 8 caracteres, solo letras en nombre y apellidos).
- Se evita el registro de usuarios con nombres de usuario o correos electrónicos duplicados usando validaciones en UserRepository.

2. Gestión de contraseñas:

- Las contraseñas no se muestran en texto plano en la interfaz; se usan campos PasswordField y solo se pueden visualizar temporalmente mediante un botón de "mostrar/ocultar".
- La contraseña se transfiere entre campos de forma controlada, evitando exposiciones accidentales en la UI.

3. Control de acceso y autenticación:

- Solo los usuarios autenticados pueden acceder a las funcionalidades principales de la aplicación (gestión de juegos, tienda, cuenta).
- El usuario autenticado se gestiona en MainWindow y se verifica antes de mostrar información sensible.

4. Manejo de errores y mensajes:

- Se muestran mensajes de error claros cuando ocurre un fallo en el registro, inicio de sesión o ejecución de juegos, evitando fugas de información interna.
- Los errores de base de datos y de ejecución se capturan y no se exponen detalles técnicos al usuario final.

5. Persistencia y acceso a datos:

- Las operaciones de base de datos usan consultas preparadas (PreparedStatement) para evitar inyecciones SQL en UserRepository y otros repositorios.
- No se exponen datos sensibles como contraseñas en logs ni en la interfaz.



6. Restricción de acciones críticas:

- No se permite ejecutar juegos si no hay un usuario autenticado o si no se ha seleccionado un juego, evitando acciones no autorizadas.

10.3 DESPLIEGUE

El software es una aplicación de escritorio desarrollada en JavaFx y Maven. Por lo tanto, el software se despliega mediante el empaquetamiento de todo el proyecto en un único fichero .jar mediante el comando: `mvn clean package`, es decir, Maven va a encargarse de reconstruir el proyecto para generar una carpeta denominada target donde se va almacenar de alguna forma todas las clases compiladas y recursos utilizados en el proyecto para finalmente, generar el proyecto en un único fichero .jar.

Por otra parte, se ha comprobado mediante el comando: `java -jar [ruta del fichero .jar]`, que la aplicación funciona correctamente, para posteriormente, configurar el fichero .exe gracias a Launch4j, una herramienta que permite convertir tu proyecto empaquetado (.jar) a una aplicación ejecutable (.exe). Para ello, se necesita el fichero .jar del proyecto y configurar los siguientes parámetros en Launch4j:

1. Basic: en este apartado configuramos el Output file (la generación del fichero .exe donde queremos guardar) y el Jar (seleccionar el fichero .jar para convertirlo a un fichero .exe).

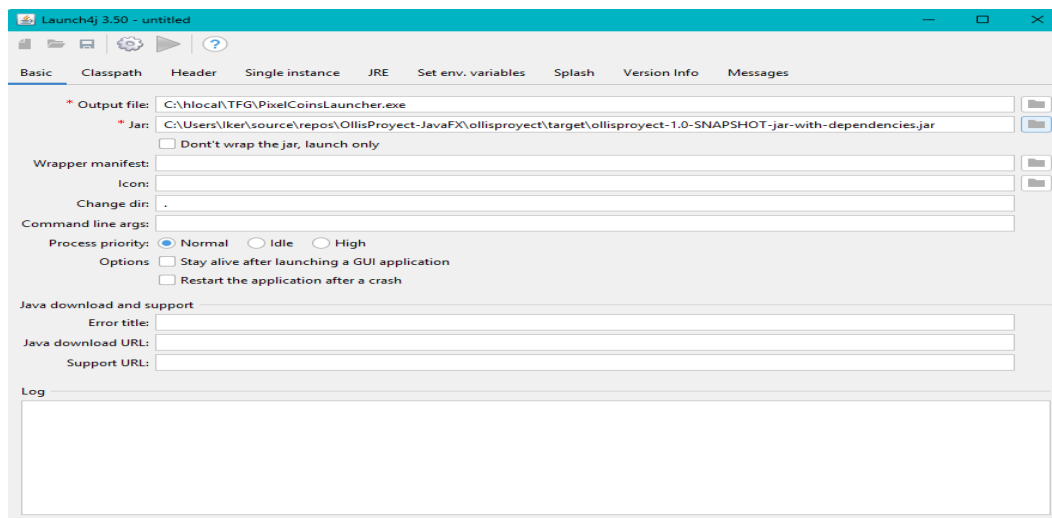


Imagen representativa de la configuración de la sección de Basic



2. JRE: configuraremos en este apartado la versión mínima de JRE a la versión 17 y pondremos en el apartado JVM options los módulos y componentes de JavaFx para que pueda compilar y ejecutar de manera correcta la interfaz de usuario mediante el siguiente comando: `--module-path [ruta de la version y los componentes y librerías de JavaFx]--add-modules javafx.controls,javafx.fxml`.

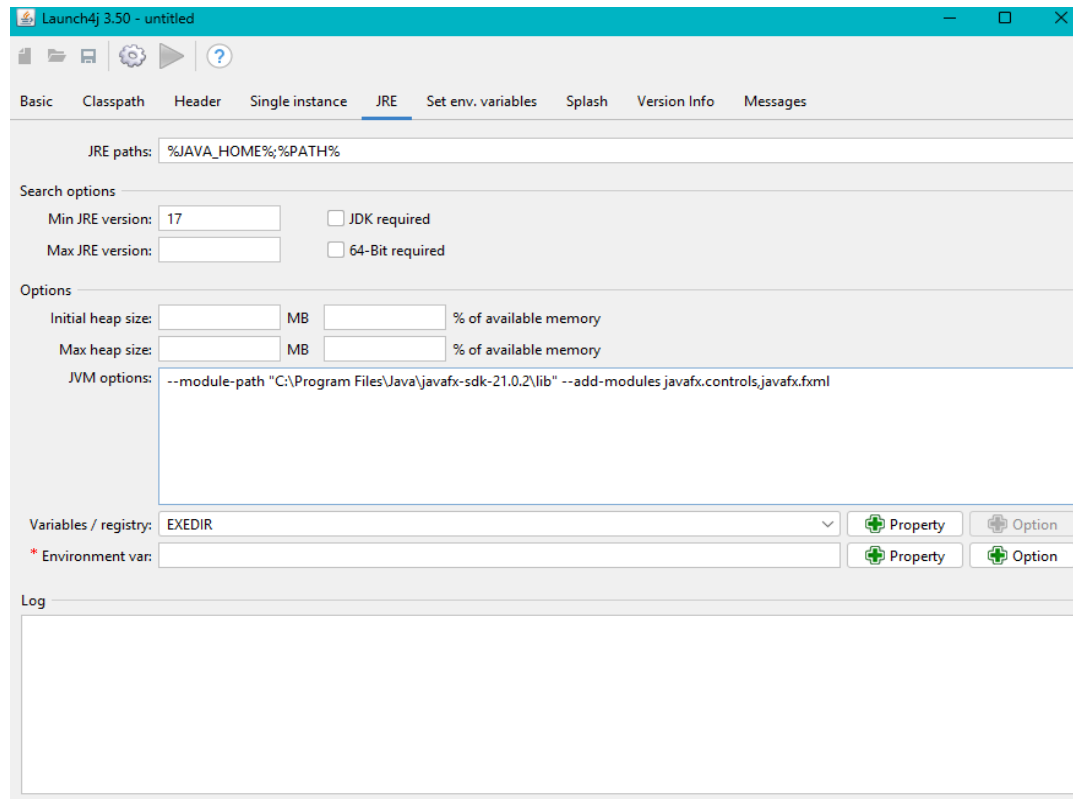


Imagen representativa de la configuración de la sección de JRE

3. Build Wrapper: para finalizar, pulsamos a la rueda de arriba para poder construir el archivo ejecutable (.exe).

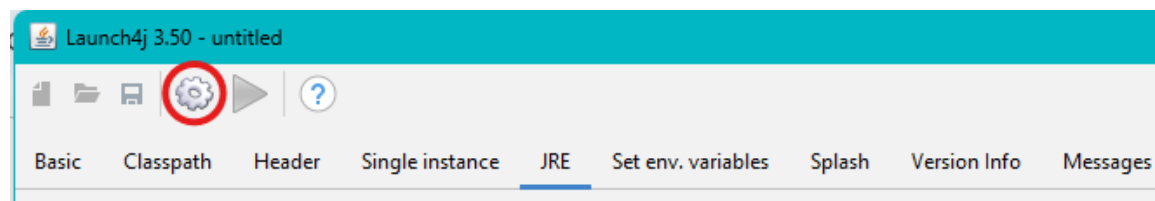


Imagen representativa donde se señala en rojo la rueda para construir el archivo ejecutable.



11. ANEXO - MANUAL DE USUARIO

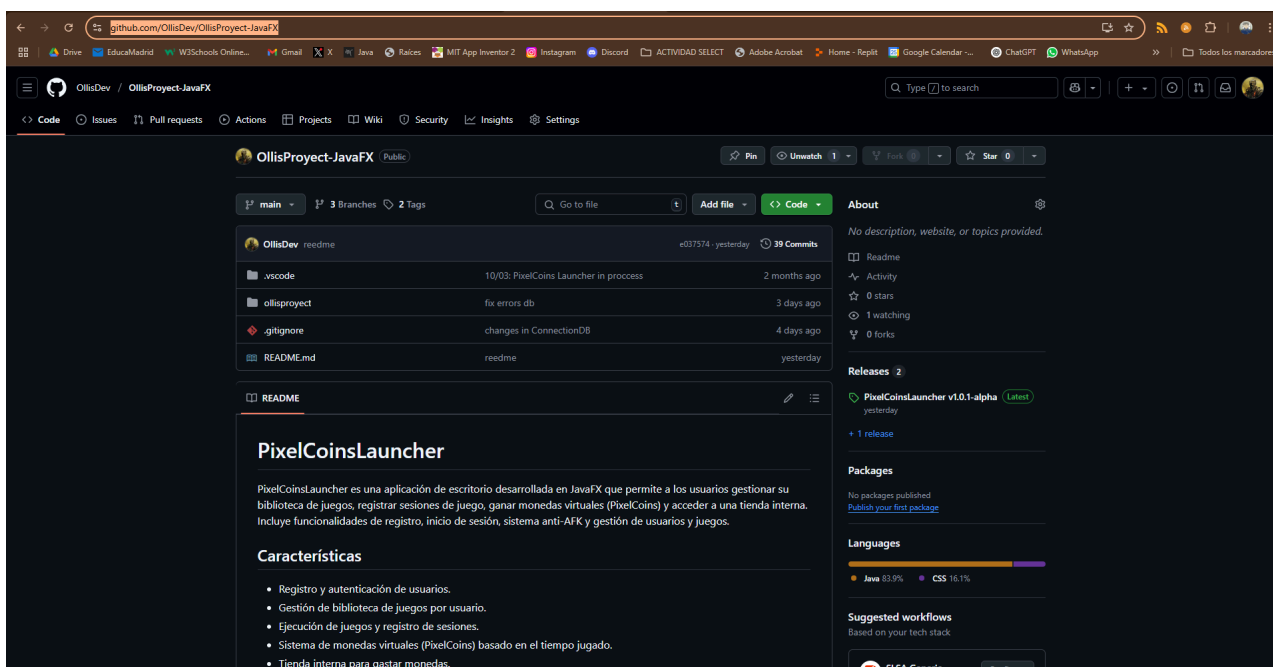
PixelCoinsLauncher es una aplicación de escritorio que te permite gestionar tu biblioteca de juegos, registrar el tiempo jugado, ganar monedas virtuales (PixelCoins) y acceder a una tienda interna. Incluye registro, inicio de sesión, sistema anti-AFK y gestión personalizada de juegos.

Requisitos previos

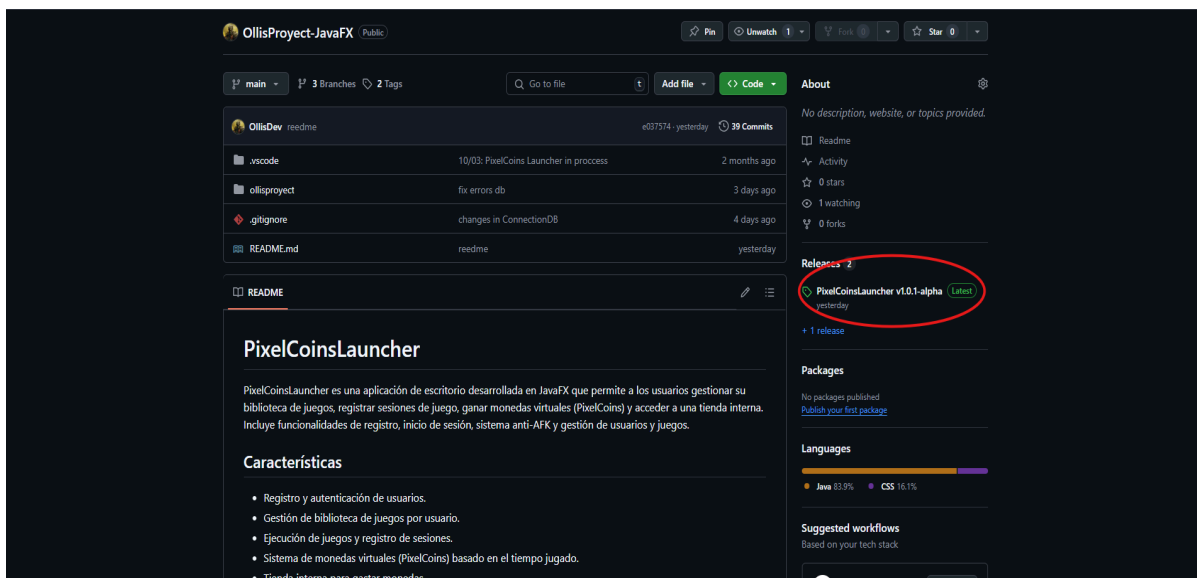
- Tener instalado Java 17 o superior.
- Conexión a Internet para el registro y gestión de datos.

Instalación

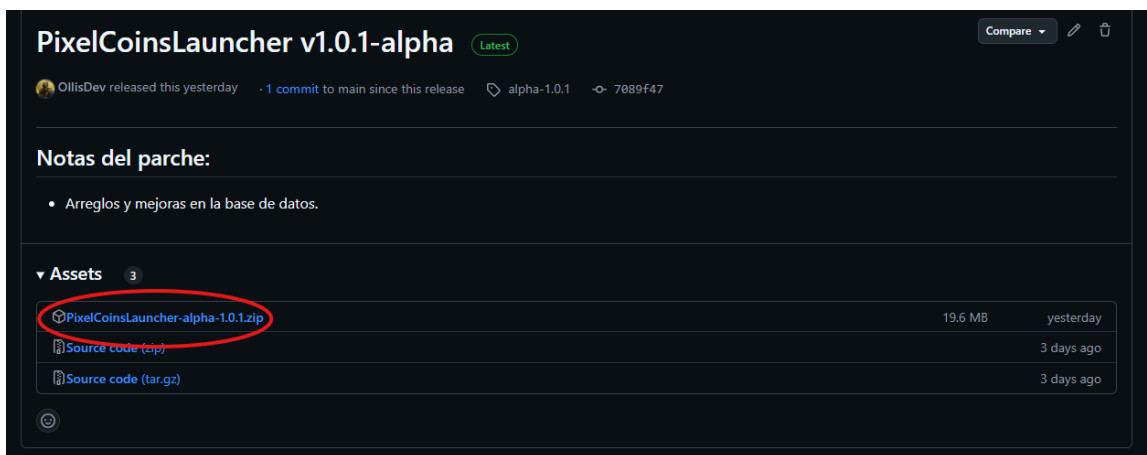
1. Copiar el siguiente enlace en el buscador del navegador:
<https://github.com/OllisDev/OllisProject-JavaFX>
2. Tendrá que salir esta página web como en la siguiente imagen:



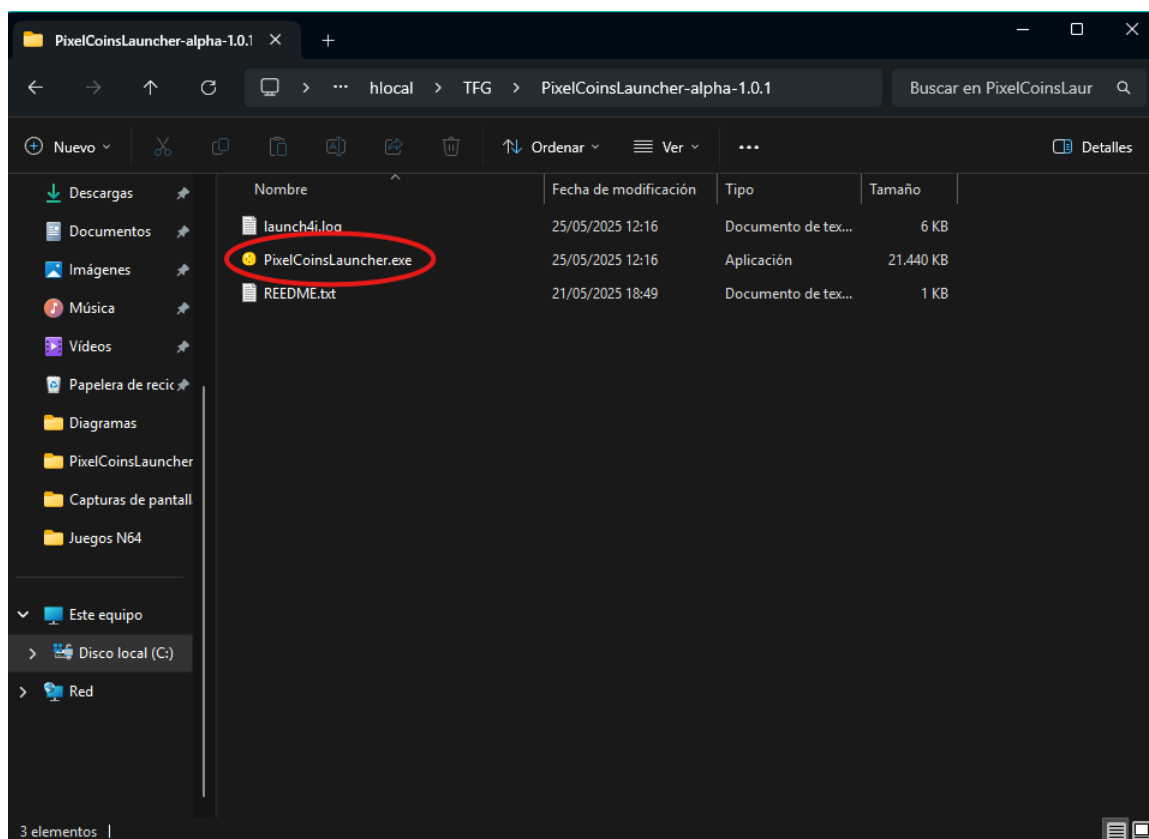
3. Ir a la sección de “Releases” y dar clic en la última versión.



4. Ya en la sección de la última versión, dar clic en el archivo comprimido (.zip) para que se descargue la carpeta del proyecto.



5. Ya descargado y descomprimido la carpeta, dentro de la carpeta del proyecto, dar clic en el archivo ejecutable (PixelCoinsLauncher.exe) para comprobar que funciona correctamente.



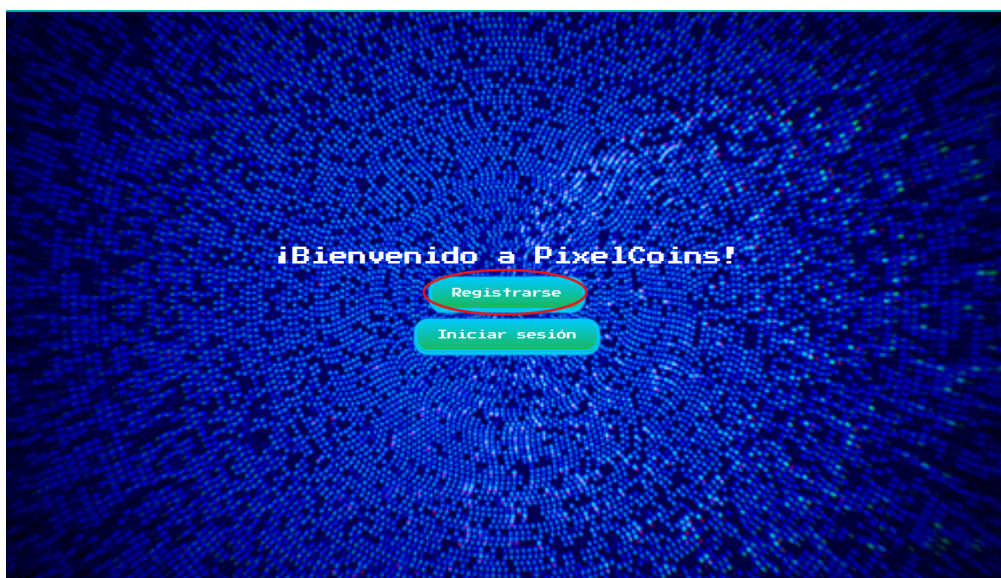


Primeros pasos

1. Pantalla principal



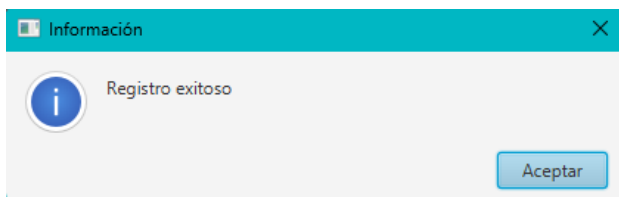
- Verás dos opciones: Registrarse o Iniciar sesión:
 - Registrarse
 - Haz clic en Registrarse.



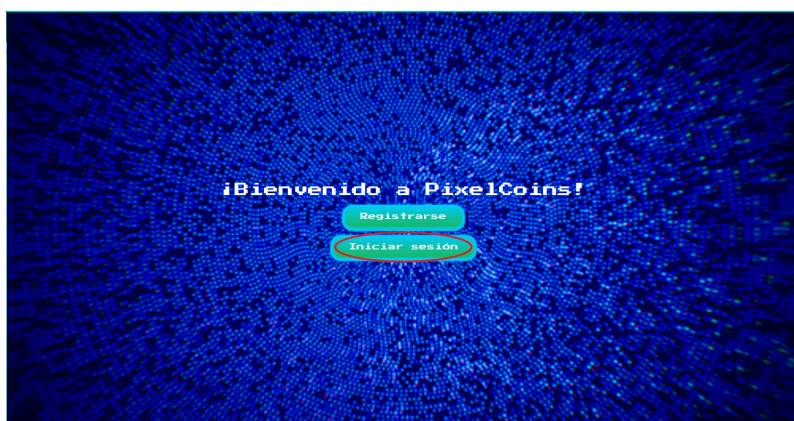


- Completa todos los campos y haz clic en “Aceptar”.

- Si los datos son correctos, verás un mensaje de registro exitoso.



- Iniciar sesión
 - Haz clic en “Iniciar sesión”.

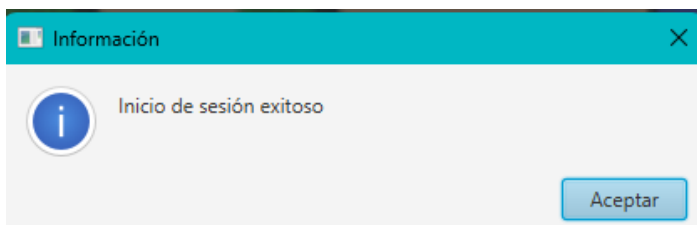




- Introduce tu nombre y contraseña con una cuenta registrada y das clic en “Aceptar”.



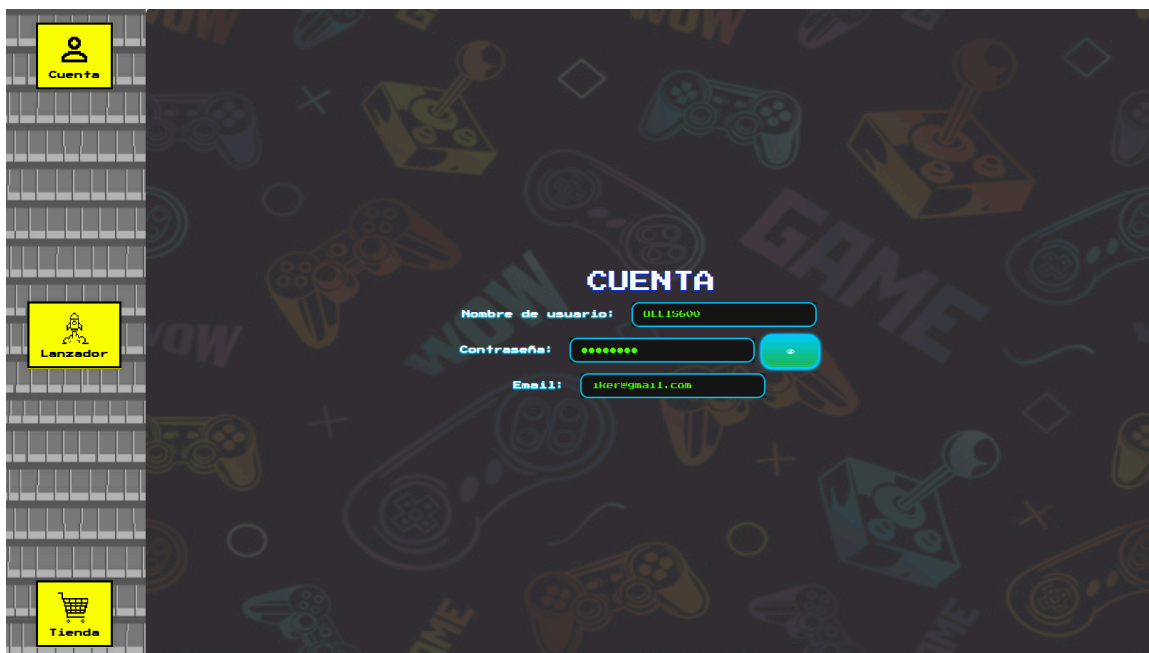
- Si los datos son correctos, accederás a la ventana principal de la aplicación.





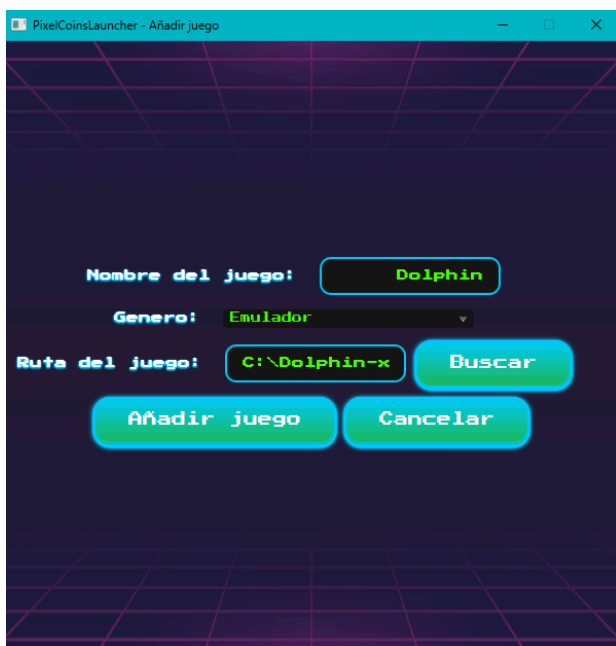
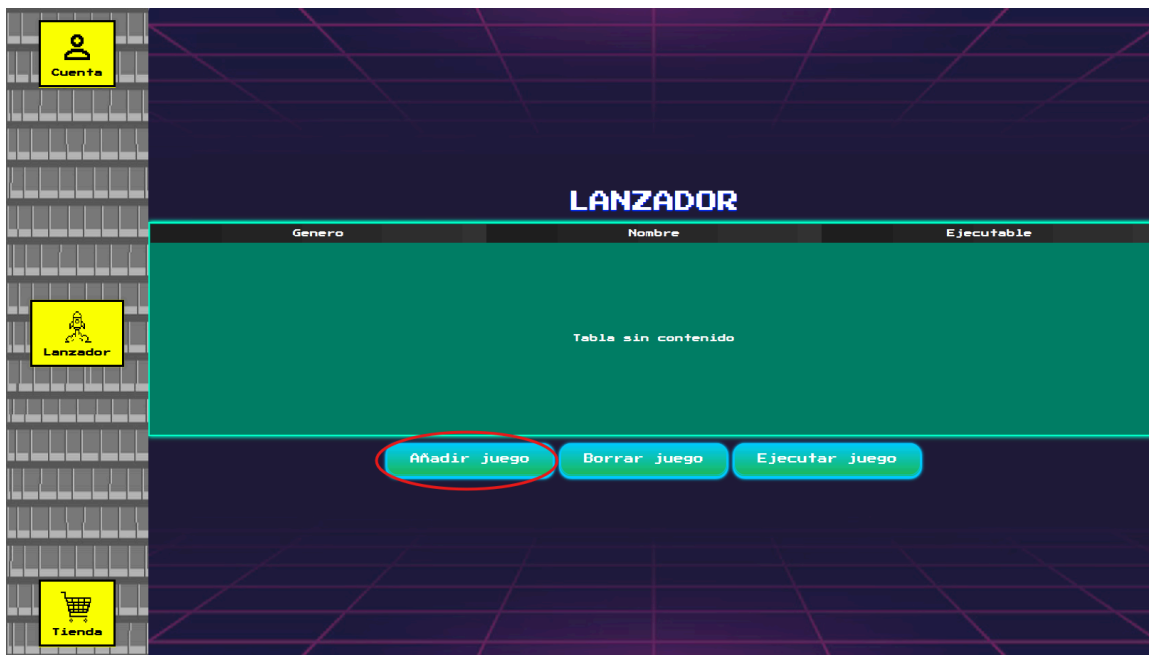
2. Funcionalidades principales

- Cuenta de usuario
 - Accede a la sección “Cuenta” para ver tus datos personales.



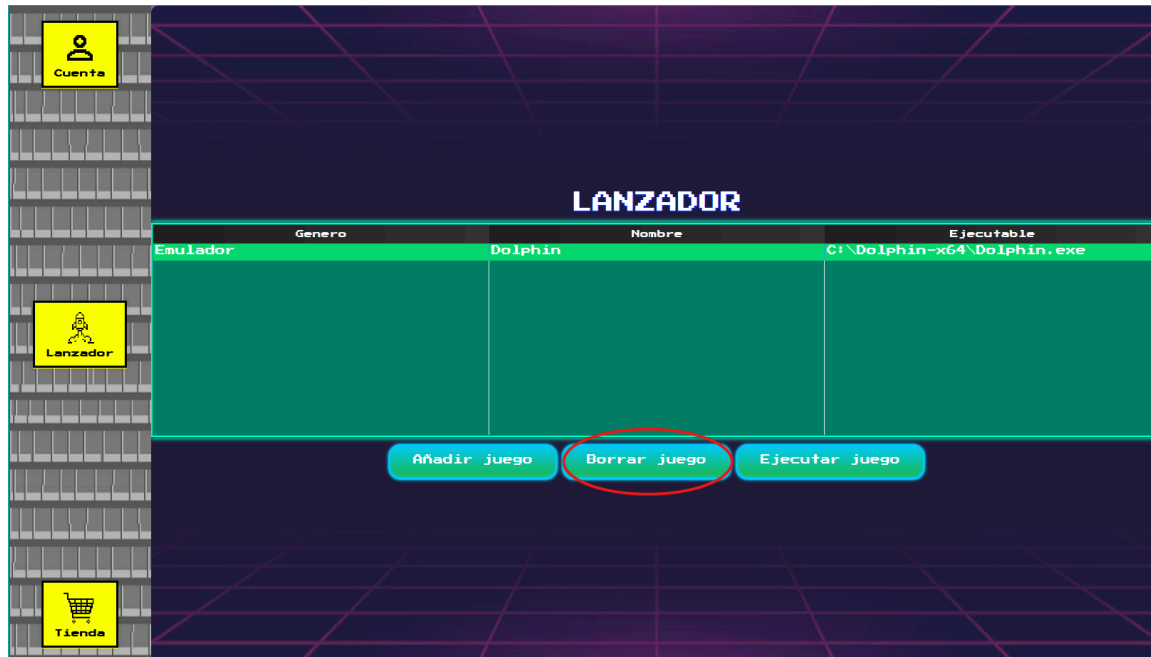


- Lanzador
 - Añadir juego: haz clic en “Añadir juego” y completa los datos y haz clic en “Añadir juego” para que salga en la biblioteca de juegos.



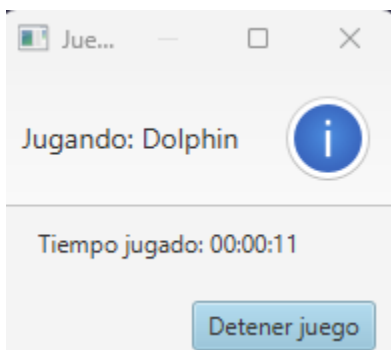
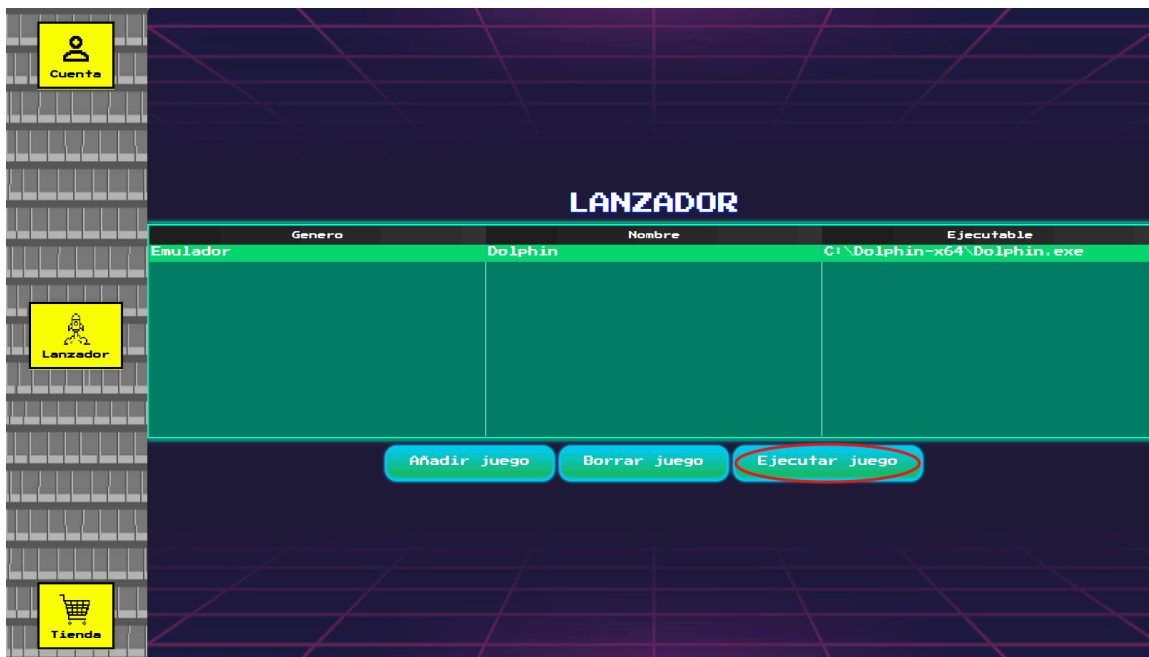


- Borrar juego: selecciona un juego que se encuentre dentro de la biblioteca de juegos y haz clic en “Borrar juego”.





- Ejecutar juego: selecciona un juego que se encuentre en la biblioteca de juegos y haz clic en “Ejecutar juego”. El sistema registrará el tiempo del jugador mediante un mensaje y cuando cierres el juego, te otorgará monedas.





Preguntas frecuentes

¿Qué hago si olvido mi contraseña?

Actualmente, debes contactar con el soporte del desarrollador para recuperar tu cuenta.

¿Puedo usar la aplicación en varios ordenadores?

Sí, pero tus juegos y monedas están asociados a tu cuenta.

¿Por qué no puedo añadir un juego?

Verifica que el nombre no esté repetido y que la ruta sea válida.

